BEGINNER EXPLAINER

ChoiceForge

How a GPU kernel can accelerate a real travel demand modeling component - from the first idea through a public ActivitySim proof.

Written for readers with no background in transportation modeling, probability, or GPU computing.

1.135x	3.654 s	6.20 GB
conservative cold component speedup	cold component time saved	verified native skim payload

Read left to right: Phase 31 cuts the conservative cold mandatory-scheduling boundary from 30.739 to 27.085 seconds, saves 3.654 seconds, and verifies every byte of a 6.20 GB GPU-ready skim store. All logsum bits and final schedules remain exact; boundaries and limits are explained inside.

Who this guide is for

This guide is for a curious high school student. You do not need to know transportation planning, probability, programming, or computer hardware. By the end, you should understand:

- what a travel demand model tries to predict;
- how a computer turns possible choices into probabilities;
- why the same calculation must be repeated millions of times;
- what CPUs, GPUs, and GPU kernels do;
- what ChoiceForge changes;
- how correctness and speed are proven on a public benchmark;
- what the completed Phase 31 persistent-skim result does and does not prove;
- why faster modeling could matter to communities.

The one-minute version

A city may want to know what could happen if it adds a bus line, changes a toll, builds housing, or closes a bridge. It cannot test every idea in the real world first, so planners use a **travel demand model**: a computer simulation of how people may decide where, when, why, and how to travel.

Modern models can represent millions of people and many possible choices. For each person, the computer scores the available choices, turns the scores into probabilities, and uses a controlled random number to select one outcome. It repeats this work again and again for trips, tours, destinations, travel modes, and times of day.

A normal processor, called a **CPU**, is very flexible but has a modest number of powerful processing cores. A **GPU** has many more, simpler cores and is good at doing the same kind of arithmetic for a huge number of records at once. A **GPU kernel** is a small program designed to run across those GPU cores.

ChoiceForge is an open-source experiment that moves a particularly repetitive travel-choice calculation to an NVIDIA GPU. Its most important idea is **fusion**: calculate utility scores, apply availability rules, compute a logsum, and make the choice in one GPU operation, without first storing an enormous table of intermediate scores.

The first synthetic test compared the GPU with a readable NumPy reference. Phase 1 added a much stronger 24-core CPU competitor. Phase 2 captured 4,477 real mandatory-tour scheduling decisions from ActivitySim, and every GPU choice matched. It also found a problem: transferring a 151.6-megabyte table made the GPU slower. Phase 3 replaced that table with 22.0 megabytes of compact ingredients and made the captured calculation 3.83 times faster than a purpose-built 48-thread CPU version. Phases 4 and 5 installed and reused the GPU path in real scheduling. Phase 6 handled real destination work. Phase 7 batched repeated setup and added nested logit on the GPU. Phase 8 moved the work to pinned current ActivitySim and a public 50,000-household run. Phase 9 then used the much larger public MTC geography. It confirmed an exact GPU destination result and also caught a scheduling mismatch before that path could be claimed as correct. Phase 11 repeated the full-geography destination experiment three times and established the supported production result: the complete 50,000-household model was 1.064 times faster with byte-for-byte identical outputs. Phase 12 built the foundation for moving the large utility equations themselves onto the GPU. Phase 13 completed the strict CPU answer key. Phase 14 generated GPU code from that same recipe and matched the CPU exactly on 30 real public-model batches. Phase 15 connected it to the real model, removed the giant intermediate table, proved a small destination-component win, and found an honest scale limit. Phase 16 added compact inputs, caches, and a published FP32 policy. Its CPU and GPU matched every checked cell, and three large public pairs proved a repeated 1.025-times trip-destination component speedup with exact modeled decisions. Phase 17 made the compiled GPU work reusable and continued it into trip mode choice. Five large public pairs improved trip destination by 1.040 times with exact

modeled decisions; the complete-model median also improved by 1.006 times, although its strict statistical gate still did not pass.

Phase 18 then kept a dependent model-shaped chain on the GPU and processed all 2.875 million public households, but its behavior equations were synthetic. Phase 19 replaced that synthetic boundary with the public calibrated auto-ownership and mandatory-tour-frequency equations. It exactly reproduced 50,000 saved household choices and 78,900 saved person choices. Median GPU modeled work was 17.840 times faster than the independent CPU replay and 12.073 times faster after including one input upload and final download. That is the strong calibrated result, although upstream location/CDAP work remained outside its boundary.

Phase 20 then solved the next structural problem. The GPU turned those 78,900 person choices into 81,983 variable-length tour rows, and every value in all 12 tour columns matched ActivitySim. It fed those exact tour IDs into six real mandatory-scheduling batches containing 15.24 million possible tour-time rows. All 81,983 GPU time choices matched ActivitySim. Tour creation was 11.496 times faster with resident data and 6.272 times faster including transfers. The scheduling kernel was 18.097 times faster with resident compact inputs and 2.935 times faster including compact transfers. ActivitySim still prepares the scheduling logsums, feasible time choices, and timetable facts on the CPU, so this is not yet a whole scheduling-component speed claim.

Phase 21 moved that missing scheduling preparation onto the GPU. It gave every person a small timetable on the graphics card, tested all 190 time choices, kept only the choices that did not collide, rebuilt the seven timetable facts used by ActivitySim, made the choice, and updated the timetable before a later tour was scheduled. Across nine measurements, this complete compact-cache boundary was 8.680 to 10.199 times faster than a compiled parallel CPU version, depending on whether primitive transfers were included. Every one of 15.24 million regenerated rows and all 81,983 time choices matched.

Phase 21 also connected the real raw network skims to the CUDA mode-logsum engine inside ActivitySim. All six live calls used the GPU with no fallback, and the final tour times were unchanged. This required matching Sharrow's fused 32-bit utility arithmetic and ActivitySim's later 64-bit nest arithmetic, because one random draw was close enough to a probability boundary to expose the difference. The 8.680-to-10.199-times result begins after a compact logsum cache exists; the raw-skim integration is a separate correctness proof, not a hidden whole-component speed claim.

Phase 22 joined those two halves. Three paired public runs ended with all 81,983 mandatory schedules unchanged, and the connected GPU path was faster in all three pairs. The middle paired speedup was 1.257 times. The GPU also found 57 choices whose random draws were so close to a probability boundary that CPU and GPU rounding could disagree. The real Sharrow arithmetic resolved only those 57 rows, using 11,400 bytes of transferred logsum data. This makes Phase 22 an exact, mostly-GPU component result - not an absolutely CPU-free model.

Phase 23 then changed the architecture. Instead of repeatedly returning to CPU-owned ActivitySim tables between decisions, it created a versioned model state that stays on the graphics card across calibrated auto ownership, mandatory-tour frequency, tour creation, tour identity linking, and timetable scheduling. A new fused kernel evaluates all 98 mandatory-frequency expressions and the final choice without building a large intermediate feature table. Three independent processes, each with nine measured repetitions, produced a middle resident speedup of **24.405 times**. Even charging one-time setup and final publication to a single run was **1.356 times faster** at the middle result. Every calibrated choice, all 12 tour columns, all 81,983 time choices, and every final timetable value matched. The runtime can also write a self-contained checkpoint, restore it, and continue computing on the GPU.

Phase 24 tackles the largest shared input: network skims. The public file contains 826 compressed matrices, but the reviewed 315-term tour-mode equations need 209 logical skim lookups. Directional views share source data, so 149 physical float32 cubes occupy 6.38 GB and fit under an 8 GiB budget. Across three independent processes, the GPU read and checked 251.76 million real-workload skim values per run with zero mismatches. The middle resident cache-layer speedup was **193.114 times** and the slowest process was still **82.323 times** faster. Even paying the entire upload once gave a middle **1.813-times** win. This is a raw-data-layer proof, not yet the complete utility/logsum stage.

Phase 25 connects that raw-data layer to the real mathematics. Six real 315-term programs now read the resident skims, score 21 travel modes, combine related modes with nested logit, and place the resulting logsums into each tour's 5-by-5

scheduling cache. The cache-placement map is compiled once and kept on the GPU too. Three fresh processes completed 15 measured replays with no changed logsum bit. The middle process time was **0.169 seconds** for all 1.21 million rows. This was **9.655 times faster** than the same process's initial live CUDA setup-and-execution path. It is not a CPU speedup or a whole model speedup. ActivitySim still creates the dense input rows, and the final live scheduler still asks the exact CPU/Sharrow path to settle 57 extremely close probability cases.

Phase 26 connects that producer directly to GPU scheduling and timetable mutation. The GPU now generates the feasible scheduling rows and their compact index, consumes the newly created logsum caches, chooses all 81,983 mandatory tour times, and updates each person's calendar inside one sealed, versioned stage. Three fresh processes completed 15 measured replays at a middle time of **0.201 seconds**, with every logsum bit and final TDD unchanged. The former 57 near-boundary cases no longer travel to a CPU resolver. They use an explicit, public-benchmark Sharrow answer map already held on the GPU; only one needs a correction, and zero boundary bytes are downloaded. This is exact for the qualified frozen benchmark, not a universal arithmetic promise for changed inputs. ActivitySim still creates the dense mode-choice leaves and coordinates before sealing them.

Phase 27 removes those prepared row arrays from the timed graph. It replaces 503.4 MB of repeated dense values and coordinates with 25.0 MB of compact facts: values shared by a whole tour, values shared by a time alternative, and small dictionaries for repeated tour-by-alternative patterns. A CUDA kernel rebuilds the exact arrays in about **0.002915 seconds**. The matched NumPy job takes **0.491203 seconds**, making this reconstruction boundary **168.52 times faster** on the GPU. The entire compact-input-to-calendar graph takes **0.205337 seconds**, only 2.23% longer than Phase 26 even though it now does the missing reconstruction. All 15 full replays keep every logsum bit and all 81,983 final times unchanged. ActivitySim still creates the dense arrays once during qualification so the compact form can be discovered and checked; removing that cold-start dependency is the next phase.

Phase 28 replaces the remaining anonymous response dictionaries with named rules. The GPU now calculates parking cost from a tour rate and duration, and calculates 14 mode-availability fields from raw resident road/transit skims and auto ownership. Three public processes and 15 complete replays finish at a middle **0.211799 seconds**, with every logsum bit and every final tour time unchanged. Compact state falls again to 20.26 MB, a **24.849-times reduction** from the removed rows. Five deliberately changed synthetic populations and skim sets exercise all 15 rules across 8,000 rows with exact, different outputs. The stronger meaning costs time: the full graph is 3.15% slower than Phase 27 because it computes real skim rules instead of looking up remembered patterns. ActivitySim still supplies dense rows before sealing as a qualification answer key, so cold raw-table generation remains unfinished.

Phase 29 takes the larger upstream step. Instead of learning compact facts by looking at the giant prepared answer rows, its compiler starts with one source row per tour, land-use facts, controlled random inputs, possible time slots, and raw network skims. It declares 57 sources for each of six programs and generates all 18 road/transit availability fields on CUDA. Parking rates now come directly from land use plus free-parking status. Across three public processes and 15 replays, the middle complete graph is **0.225311 seconds**; every logsum bit and all 81,983 final times still match. Five changed raw-table populations (10,000 tours) and five changed skim worlds (8,000 rows) also pass. The compact state is 24.85 MB, **20.259 times smaller** than the removed rows. The qualification harness still runs the old dense path separately as an answer key and to expose the compiled utility interface, so the roughly 30.759-second cold ActivitySim run has not disappeared yet.

Phase 30 removes that production dependency. The GPU interface is now built directly from a reviewed equation recipe, declared raw-table types, scalar settings, controlled random draws, and raw skim-cube descriptions. The old dense preprocessor is run only in a separate proof process. Across three fresh public processes, all 1,210,124 dense rows are avoided, all 15 resident replays are exact, and all 81,983 final times match. A byte-level hash test also proves that the native and legacy paths generate identical six logsum arrays. The middle resident time is **0.226712 seconds**. The middle cold run is **30.739 seconds**, essentially unchanged because loading 6.452 GB of network skims is still the largest startup cost. This phase is a major independence and replication result, not a major new cold speedup.

Phase 31 attacks the startup cost that Phase 30 exposed. It converts the needed network measurements into a checked, GPU-ready file once, then skips Sharrow's 6.452 GB dataset in every live run. Three fresh processes keep every logsum bit and all 81,983 schedules exact. Including setup for the small on-device boundary map, the middle cold component time

falls from **30.739 seconds to 27.085 seconds**: **3.654 seconds saved**, **11.887% lower**, and **1.135 times faster**. This is the first material cold-start gain since the native runtime work began; it applies to this resumed mandatory-scheduling component, not the whole regional model.

1. What is travel demand modeling?

Imagine a region with homes, schools, offices, stores, roads, sidewalks, rail lines, and buses. Transportation planners ask questions such as:

- How many people might travel during the morning rush?
- Where might they go?
- Will they drive, walk, bike, take transit, or share a ride?
- Which roads or transit services might become crowded?
- Who benefits from a project, and who may be left out?
- How might results change if fuel prices, fares, jobs, or housing change?

A travel demand model does not read minds or predict one person's future with certainty. It creates a plausible regional picture by applying observed patterns and mathematical rules to many simulated people.

People, households, and zones

Models usually begin with a **synthetic population**. These are made-up records that resemble the region's real population without being a list of actual named residents. A record may describe a household's size, income, vehicles, workers, and students.

The region is divided into geographic areas called **zones**. A zone may contain homes, jobs, schools, shops, or transit stations. The model also needs information about travel between zones, such as distance, driving time, transit time, cost, and whether a route is available. These travel measurements are often called **skims**.

Activities, tours, and trips

People travel because they want to do activities. Going from home to school is a trip. Going from home to school, then to a store, then home is often treated as a **tour**: a chain of trips that begins and ends at the same anchor, usually home or work.

An activity-based model may answer a sequence of linked questions:

```
Who is traveling?
    -> What activity are they doing?
    -> Where will they go?
    -> What time will they travel?
    -> Which travel mode will they use?
    -> Which trips and tours result?
```

These decisions affect one another. A faraway destination may make walking unrealistic. A late return time may make transit unavailable. A household with one vehicle and two workers may face competition for the car.

2. How does a model represent a choice?

Suppose Maya must choose among walking, taking the bus, and riding in a car. The model gives every option a **utility score**. Utility is not electricity here. It is a numerical score for how attractive an option is according to the model.

A simplified utility equation looks like this:

```
utility = constant + (time x time weight) + (cost x cost weight) + other terms
```

A negative time weight means longer trips are less attractive. A negative cost weight means expensive trips are less attractive. The weights are estimated from travel surveys or other observations. Different people can receive different scores because income, age, vehicle access, location, and other facts may matter.

Availability comes first

Not every option is possible. Walking may be unavailable for a very long trip. A transit route may not exist. A person may not have access to a car. The model applies an **availability rule** so an impossible option receives no chance of being chosen.

From scores to probabilities

Raw utility scores are not probabilities. A common model uses this rule:

```
probability of option i = exp(utility i) / sum of exp(utility for every available option)
```

`exp` is a mathematical function that turns higher scores into much larger positive numbers. Dividing by the total makes all probabilities add to 1, or 100 percent.

Here is a small example:

Option	Utility score	exp(score)	Probability
Walk	0	1.000	9.0%
Bus	1	2.718	24.5%
Car	2	7.389	66.5%

The car is most likely, but it is not guaranteed. This matters because real people with similar situations do not all make the same decision.

Turning probabilities into one choice

The model takes a random draw between 0 and 1. It lays the probabilities end to end:

```
Walk: 0.000 to 0.090
Bus:  0.090 to 0.335
Car:  0.335 to 1.000
```

If the draw is 0.600, the model chooses car. If it is 0.200, it chooses bus. This is called **inverse cumulative distribution sampling**, but the important idea is simple: larger probability ranges are more likely to catch the draw.

ChoiceForge does not invent random numbers inside the GPU. The calling model supplies them. That design lets the CPU and GPU use the exact same draws, which makes fair comparison possible and preserves ActivitySim's rules for repeatable simulations.

What is a logsum?

A **logsum** summarizes how good the full set of choices is:

```
logsum = log(sum of exp(utility for every available option))
```

In the example, the logsum is about 2.407. If a fast, affordable new bus service improves one option, the logsum rises. Models often pass this accessibility-like value into later decisions.

Computers must calculate logsums carefully. Very large utility scores can make `exp(score)` overflow, like a calculator displaying an error for a number that is too large. The standard safe method subtracts the largest utility before exponentiating and adds it back afterward. This is called **stable logsum-exp**. ChoiceForge uses this method.

3. Why does this become a computing problem?

The three-option example is tiny. A regional model may have:

- millions of people, households, tours, or trips;
- dozens of alternatives for mode or time choices;
- hundreds or thousands of possible destinations;
- many variables in every utility equation; and
- dozens of model steps that repeat similar calculations.

If one million choosers each consider 32 alternatives, the model evaluates 32 million chooser-option combinations in just one component. Destination choice can be much larger.

A straightforward program may create a rectangular **utility matrix** with one row per chooser and one column per alternative. For one million choosers and 32 alternatives stored as 32-bit numbers, that table alone uses 128 megabytes. The program may then create more tables for exponentials, probabilities, or cumulative probabilities. Moving and storing these intermediate values can take as much time as the arithmetic.

4. CPU, GPU, and kernel in everyday language

A **CPU** is the general-purpose brain of a computer. Think of it as a small team of highly trained chefs. Each chef can handle complicated instructions, switch tasks quickly, and make decisions.

A **GPU** was designed to calculate many screen pixels at once. Think of it as a very large kitchen crew. Each worker is less independent, but thousands of workers can perform the same short recipe on different ingredients at the same time.

A **GPU kernel** is that short recipe. It says what one group of GPU threads should calculate. Launching a kernel sends many copies of the recipe across the data.

The GPU is not automatically faster. Three costs matter:

1. **Transfer:** data may need to cross from ordinary computer memory to GPU memory and back.
2. **Launch overhead:** starting a GPU kernel takes a small amount of time.
3. **Parallel work:** the job must be large and regular enough to keep many GPU cores busy.

Small or irregular jobs may be faster on the CPU. A good GPU design must move enough connected work together and avoid unnecessary transfers.

5. The main ChoiceForge idea: fuse the pipeline

A less efficient pipeline might do this:

```
CPU creates inputs
    -> GPU computes and stores every utility
    -> another operation reads utilities and computes probabilities
    -> another operation reads probabilities and selects choices
    -> results return to CPU
```

ChoiceForge's fused linear kernel aims to do this:

```
chooser features + alternative weights + constants + availability + random draw
    -> one fused GPU kernel
    -> selected alternative + logsum
```

Inside the kernel, one GPU block handles one chooser. Threads in the block work on the alternatives together. They calculate utility scores, ignore unavailable options, find the maximum score, form safe exponential weights, sum them, select the option containing the supplied random draw, and return only the useful outputs.

The intermediate chooser-by-alternative utility matrix is never written to large global GPU memory. Temporary values stay close to the GPU cores in faster **shared memory**. This reduces memory traffic and storage.

The milestone-one kernel supports:

- a fixed set of 1 to 1,024 alternatives;
- linear utility equations using 32-bit floating-point numbers;
- availability rules;
- stable logsums;
- externally supplied random draws; and
- exact comparison of selected alternatives with a CPU reference.

It does not yet support all of ActivitySim's expression language, estimation mode, or destination sets with thousands of alternatives. Phase 7 supports the canonical MTC 21-mode nested-logit tree, but not every irregular ActivitySim choice model.

6. How the prototype is kept honest

Speed is useless if the model silently changes its answers. The project therefore starts with a readable NumPy CPU implementation called the **reference** or **oracle**. The GPU result is checked against it.

The validation rules are:

- give the CPU and GPU identical inputs and identical random draws;
- preserve the same alternative order and boundary rules;
- require every selected alternative to match exactly;
- measure the numerical difference between CPU and GPU logsums;
- test unavailable alternatives and invalid rows; and
- report mismatches instead of hiding them.

Small floating-point differences are normal because parallel GPU operations may add numbers in a different order. It is like adding a long list of rounded decimals from left to right versus pairing them first: the last digit can differ. The selected choices still must match, and logsum differences must stay within an explicit tolerance.

The Python 3.11 ActivitySim and CUDA integration environment now passes 95 tests. These include exact comparison with ActivitySim's real Numba choice function, compact expression compilation, segmented destination batches, categorical flags, nested-logit validation, current-version fallback forwarding, real scheduling integration, GPU tests using 33 and 190 alternatives, a safe expression interpreter, skim-table adapters, shadow checks, the strict CPU reference, the generated strict CUDA evaluator, the explicit FP32 policy used for Phase 16, Phase 17's schema-safe plan and workspace reuse, and Phase 18's fail-closed GPU state, stable random draws, deterministic partitions, and ordered aggregation.

7. What was benchmarked?

A **benchmark** is a controlled timing experiment. ChoiceForge's first benchmark uses synthetic, meaning computer-generated, data. Each test has:

- 32 alternatives;

- 16 features per chooser;
- 10,000, 100,000, or 1,000,000 choosers; and
- 32-bit floating-point calculations.

It compares three timings:

1. **NumPy materialized:** the readable CPU reference creates the utility matrix.
2. **GPU including transfers:** inputs move to the GPU, the kernel runs, and outputs move back.
3. **GPU resident:** needed arrays are assumed to already be in GPU memory.

The GPU-resident number shows the potential of a future pipeline that keeps related model steps on the GPU. The transfer-inclusive number is the more conservative measure for a single isolated call.

All numbers below are medians after warm-up on an NVIDIA RTX A4000 with 16 GB of memory.

Choosers	NumPy CPU	GPU incl. transfers	GPU resident	End-to-end speedup	Resident speedup
10,000	5.156 ms	0.845 ms	0.361 ms	6.10x	14.26x
100,000	50.614 ms	3.587 ms	1.037 ms	14.11x	48.79x
1,000,000	499.375 ms	36.610 ms	8.118 ms	13.64x	61.51x

For all three sizes:

- choice mismatches: **0**;
- maximum logsum error: less than **0.000001**; and
- the fused kernel avoided allocating the global utility matrix.

At one million choosers, this original comparison was about 13.64 times faster. But NumPy was built as a readable correctness reference, not the strongest possible CPU competitor. Phase 1 therefore added a fused Numba implementation that spreads choosers over all 24 physical CPU cores.

Phase 1: comparison with the strongest CPU baseline

For the original 32-alternative, 16-feature shape, the 24-core fused CPU was much stronger:

Choosers	Best 24-core CPU	GPU incl. transfers	Transfer-inclusive result	GPU resident
10,000	0.390 ms	1.260 ms	GPU is 0.31x as fast	0.810 ms
100,000	5.560 ms	4.170 ms	GPU is 1.33x faster	1.190 ms
1,000,000	59.260 ms	34.520 ms	GPU is 1.72x faster	7.110 ms

This result is more realistic and less dramatic. The GPU loses on the smallest job because transfer and launch costs dominate. Its advantage grows with the amount of work.

Phase 1 also tested a shape based on mandatory tour scheduling in prototype MTC. It has 190 time alternatives and 69 feature rows:

Choosers	Best 24-core CPU	GPU incl. transfers	Speedup	GPU resident	Choice mismatches
10,000	10.760 ms	2.560 ms	4.19x	0.970 ms	0
100,000	109.660 ms	20.380 ms	5.38x	6.730 ms	2

The wider calculation gives each transferred row much more arithmetic, which suits the GPU. The two differences at 100,000 rows occurred when random draws were less than three ten-millionths from a probability boundary. NumPy chose one option while both fused Numba and CUDA chose the immediately following option. This tiny but important result means the project needs a clear 64-bit or mixed-precision rule before claiming exact ActivitySim equivalence.

The wider test also discovered and fixed a real GPU synchronization bug. The original 32 alternatives fit inside one group of 32 GPU threads, called a warp. With 190 alternatives, several warps shared temporary memory. One warp could reuse that memory before another had finished reading it. A new synchronization barrier fixes the race, and tests above 32 alternatives prevent it from returning.

Phase 2: replaying real ActivitySim scheduling data

Phase 1 copied the *shape* of tour scheduling, but its people and feature values were generated by a random-number program. Phase 2 uses real model records from ActivitySim's public `prototype_mtc` example.

ActivitySim's **mandatory tour scheduling** step chooses when work and school tours begin and end. For example, one option might leave at 8 a.m. and return at 5 p.m. The utility equation considers 52 to 59 active terms, including:

- the traveler's type, income, and work or school situation;
- the proposed start time, end time, and duration;
- whether another tour already occupies part of the day;
- a mode-choice logsum describing the quality of available travel modes; and
- constants that make some departure, arrival, and duration ranges more or less attractive.

The capture program observes ActivitySim at a documented internal boundary. It records the evaluated expression terms, feasible alternatives, coefficients, probabilities, random draws, and final selected positions. It does not change ActivitySim's rules or generate new draws.

The public sample produced six mandatory scheduling batches: first and second tours for work, school, and university students. Together they contain 4,477 tour choices. The largest batch contains 3,381 first work tours and 642,390 feasible chooser-alternative rows.

Did the answers stay the same?

Yes. All 4,477 GPU choices matched ActivitySim exactly. A separate 64-bit replay of ActivitySim's probability tables also had zero mismatches. The largest batch's maximum utility difference was less than 0.00000044. In the full model output, every tour's selected time alternative, start time, and end time matched the earlier warm Sharrow run.

Was the GPU faster?

The answer depends on where the input data begins:

Copies of the real batch	Choosers	CPU	GPU including transfers	GPU already resident	Resident speedup
1	3,381	8.241 ms	15.545 ms	0.961 ms	8.57x
2	6,762	20.346 ms	32.298 ms	1.326 ms	15.34x
4	13,524	43.035 ms	62.427 ms	2.242 ms	19.20x

For the larger rows, Phase 2 repeats captured real tours and their original draws. It does not invent synthetic travelers. Repeating records is a throughput test, not a claim that the public model contains more households.

The resident kernel clearly wins. The transfer-inclusive path clearly loses. The reason is that the current lowerer expands the expressions into a large table of term values: 151.6 megabytes for the native largest batch. Sending that table across the computer's PCIe connection takes much longer than the GPU arithmetic.

An analogy is a very fast bakery on the other side of town. Baking takes one minute, but delivering every separate ingredient takes fifteen minutes. Buying a faster oven will not solve the delivery problem. The next design must send compact ingredients once, prepare more of them inside the GPU, and keep them there for connected model steps.

What has Phase 2 proved?

It has proved that the ragged scheduling kernel can process the real ActivitySim alternative sets with exact choices, and that its resident computation is faster than a strong CPU boundary on this machine. It has also disproved the idea that simply adding a GPU call around an expanded term table makes the isolated component faster.

At that Phase 3 boundary, it had not yet proved that the entire mandatory scheduling component or full ActivitySim model ran faster. Phases 4 through 6 later added those wider proofs. Stateful timetable expressions and other ActivitySim front-end work still run on the CPU.

Phase 3: sending compact ingredients instead of finished terms

Phase 2 sent one value for every expression and every feasible alternative. Many values repeated the same traveler fact or the same time-alternative fact thousands of times. It was like shipping 59 finished ingredient bowls for every possible schedule.

Phase 3 sends a compact package instead:

- traveler facts are stored once per tour chooser;
- start, end, and duration are stored once for each of 190 time alternatives;
- each feasible row carries a small alternative ID;
- only the mode-choice logsum and seven timetable-dependent values stay row-specific; and
- ActivitySim still supplies the exact random draw.

The compact package for the largest real batch is 22.046 megabytes instead of 151.604 megabytes, an 85.46 percent reduction.

What does the compiler do?

A **compiler** translates instructions from one form into another. Here it reads the real ActivitySim expressions, checks that every operation and column is supported, and writes matching CUDA arithmetic into the kernel. It supports the scheduling model's numbers, arithmetic, comparisons, and Boolean combinations such as "A and B." Unknown names and unsupported function calls cause a clear error instead of a silent wrong answer.

The same expressions also generate a strong CPU version compiled by Numba and spread across 48 CPU threads. This makes the comparison much harder and fairer than comparing the GPU only with a simple Python loop.

Copies of the real batch	Choosers	Compact input	48-thread CPU	GPU including transfers	GPU resident	Inclusive speedup
1	3,381	22.046 MB	9.481 ms	2.478 ms	0.181 ms	3.83x
2	6,762	44.091 MB	18.443 ms	4.681 ms	0.271 ms	3.94x
4	13,524	88.179 MB	37.132 ms	9.348 ms	0.410 ms	3.97x
8	27,048	176.355 MB	69.735 ms	17.787 ms	0.709 ms	3.92x

All six mandatory-scheduling batches still have zero choice mismatches. The largest utility difference from the Phase 2 calculation is 0.0000229, and the largest GPU-versus-CPU logsum difference is 0.00000293.

Phase 3 proves transfer-inclusive superiority at this captured kernel boundary. It still does not time the work needed to build the compact tables inside ActivitySim, calculate mode-choice logsums, update the timetable, or map results back into pandas tables. Those steps belong in the next complete-component test.

Phase 4: putting the GPU inside the real ActivitySim component

Phase 4 performs that complete-component test. ActivitySim now has an explicit setting that chooses either its normal scheduling calculation or ChoiceForge. It is not a hidden test hook. ActivitySim still decides which alternatives are possible, calculates mode-choice logsums, supplies the controlled random numbers, updates each person's timetable, and writes the output tables.

The first integrated version was correct but slower. A profile showed that the GPU work was not the problem. Repeatedly asking seven timetable questions for 642,390 rows took about 4.03 seconds. A day in this model has only 21 time slots, so ChoiceForge now answers those questions once on a small traveler-by-time-slot grid and looks up the required answers. That reduced the largest timetable stage to about 0.331 seconds.

Two three-trial experiments measured the result:

Test	Cached Sharrow median	ChoiceForge median	Speedup	Schedule mismatches
Normal full-model runs	5.515 s	4.925 s	1.120x	0
Same saved checkpoint before scheduling	8.599 s	8.014 s	1.073x	0

The timer includes the work that Phase 3 left outside: mode-choice logsums, timetable calculations, compact packing, ActivitySim's random-number call, transfers, GPU work, result mapping, and timetable updates. It even includes fresh-process GPU setup. All three normal ChoiceForge runs produced exactly the same 9,806 final `tdd`, start, end, duration, destination-logsum, and mode-choice-logsum values as the cached-Sharrow reference.

Why is the complete-component speedup 1.120x instead of 3.83x? The 3.83x number describes the calculation ChoiceForge directly replaces. The complete component also performs substantial mode-choice logsum and table work that remains the same in both versions. Speeding up one part of a larger job gives a smaller improvement to the whole job. This is an example of **Amdahl's law**: total speedup is limited by the work that was not accelerated.

Phase 5: one backend for the scheduling family

Mandatory tours are only one kind of tour. A **non-mandatory tour** might be shopping, eating out, escorting someone, or recreation. A **joint tour** is shared by members of a household. An **at-work subtour** starts and ends at a workplace, such as going out for lunch. These components use related choice math, but they do not have identical columns or timetable rules.

Phase 5 taught the compiler to understand those differences without creating three separate GPU systems. It can turn text categories such as "escort" into compact yes-or-no columns, recognize both "equals" and "does not equal," and discover whether a timetable row belongs to a person or a tour. Calculations that are safe and repetitive go into the fused GPU kernel. Special state-changing timetable work remains explicit and checked.

Three complete-model trials measured all four ActivitySim scheduling timers:

Scheduling component	Cached Sharrow median	ChoiceForge median	Speedup
Mandatory	5.515 s	5.077 s	1.086x
Joint	0.671 s	0.504 s	1.331x
Non-mandatory	1.560 s	0.550 s	2.836x
At-work subtour	0.309 s	0.202 s	1.530x
All four together	**8.045 s**	**6.325 s**	**1.272x**

The four-component scheduling family uses 21.4 percent less time, saving a median 1.720 seconds. This was not caused by one lucky run: the slowest ChoiceForge trial was still faster than the fastest cached-Sharrow trial. Non-mandatory scheduling improves most because it contains a large amount of repeated choice work but does not carry the same expensive mode-choice-logsum work as mandatory scheduling.

Correctness is checked more strongly than before. For every Phase 5 trial, the final accessibility, households, joint-tour participants, land use, persons, tours, and trips files have exactly the same SHA-256 fingerprints as the cached-Sharrow reference. A SHA-256 fingerprint is a long number made from every byte in a file; changing even one character almost certainly changes the fingerprint. This covers 9,806 tour rows and 23,583 trip rows, not just a few selected columns.

There was an important Phase 5 limit. Its separately run sessions did not prove an all-model gain because unrelated run-to-run variation was larger than the scheduling savings. Phase 6 later closed that evidence gap with an alternating A/B experiment.

8. A negative result that teaches an important lesson

The project also tested only ActivitySim's final probability-sampling operation for 100,000 choosers and 32 alternatives.

Method	Time	Result compared with warm ActivitySim Numba
ActivitySim Numba on CPU	2.515 ms	baseline
GPU including transfers	2.591 ms	0.97x, slightly slower
GPU resident	0.212 ms	11.86x faster

There were zero choice mismatches. However, after paying to transfer the probability table, the isolated GPU call was slightly slower than ActivitySim's warm CPU function.

This is useful evidence, not a failure to hide. It shows that sending only the final tiny step to the GPU is a poor design. The strong performance idea depends on fusing utility evaluation, logsum calculation, and sampling, then keeping data on the GPU across connected steps.

9. How ActivitySim fits in

ActivitySim is a popular open-source activity-based travel modeling framework written in Python. ChoiceForge is designed as a possible specialized computing backend for parts of ActivitySim, not as a new travel demand model that replaces its behavioral rules.

The project integrated ChoiceForge with ActivitySim 1.4 and ran the public 25-zone prototype MTC example from start to finish:

- 5,000 households;
- 8,212 persons;
- 34 completed model steps;
- 70.568 seconds total runtime; and
- 580.8 MB peak unique memory.

That early 70.568-second run was only a framework baseline. Phase 4 later ran the full model with ChoiceForge handling mandatory scheduling, and Phase 5 expanded it to all four tour-scheduling components.

ActivitySim can also use **Sharrow**, an optimized expression system. Its compile run eventually completed in 27.3 minutes and used about 15 GB of unique memory. That compilation is not a fair production timing. After the cache existed, three warm runs took 61.650, 59.119, and 61.790 seconds, for a median of 61.650 seconds. Their final household, person, tour, and trip files were identical.

The warm profile showed where time is spent. Trip destination used 25.3% of total time. Mandatory tour scheduling used 9.1%, and trip scheduling used 8.4%. Mandatory scheduling is a practical first integration target, but it cannot reduce total runtime by 10% on its own because it is only 9.1% of the model. A reusable scheduling backend should target several scheduling components together.

Phase 6: making destination work large enough

Trip destination asks where an intermediate stop happens. For every candidate stop, the model also estimates how attractive the travel from the trip origin to the stop would be and how attractive the remaining travel to the main tour destination would be. Those are the two **directions** called OD and DP.

The first GPU replay exposed 30 small groups, separated by trip number and purpose. Sending each group to the GPU separately was a bad bargain: the GPU took about 15.032 milliseconds including transfers, while the CPU needed only 2.646 milliseconds. Think of sending 30 nearly empty delivery trucks instead of loading one truck.

ChoiceForge then packed all 30 groups into one segmented kernel launch. A segment label tells the kernel which purpose's coefficients to use, while offsets tell it where each traveler's uneven list of candidate places begins and ends. The packed job contained 3,971 choosers and 53,927 candidate rows. It took 0.847 milliseconds on the GPU including transfers versus 1.241 milliseconds for a strengthened, batched Numba CPU version: **1.464 times faster**. All 3,971 chosen positions matched, and logsums differed by at most about nine millionths.

The crossover test explains when to use it. At 28,570 rows the GPU was slower. At 39,706 rows it was faster. On this computer, this shape should be packed to roughly 35,000 rows before GPU dispatch. That threshold must be remeasured on other computers.

The first proven whole-model improvement

The live ActivitySim loop still asks for one small purpose group at a time, so turning on the new GPU sampler there would make it slower. Phase 6 instead removed duplicated work from the much larger directional-logsum calculation. It preserved ActivitySim's OD random draws, then its DP draws, but processed the shared deterministic work together.

Six fresh full-model runs alternated the old and new paths in the order A1/B1/A2/B2/A3/B3. Trip destination fell from a 16.307-second median to 14.238 seconds, **1.145 times faster**. All 34 model steps fell from 59.209 to 56.264 seconds, **1.052 times faster**. Every new-path component run beat every old-path component run. Seven final result files were byte-for-byte identical in every trial.

This distinction matters: the 1.464x number proves the packed CUDA kernel boundary; the 1.145x number proves the complete destination component; and the 1.052x number proves this complete example model. They are not interchangeable.

Phase 7: stop repeating setup, then accelerate the actual nest

Phase 7 asked what was still consuming the 14.238 seconds. ActivitySim repeated a 70-step preparation recipe once for each of ten trip purposes, even though most of that recipe was the same. ChoiceForge now puts the purposes for one trip number into one larger worksheet. The recipe runs three times - once for trip number 1, 2, and 3 - instead of 30 times. Sampling and final decisions remain separate, so each traveler keeps the same random draws and behavioral rules.

The next boundary is called **nested logit**. Imagine grouping 21 travel modes into folders: driving modes, walking/biking, transit, and ride-hailing. Some folders contain smaller folders. Nested logit combines the scores inside each folder and then combines the folders. It represents that two similar choices, such as two transit services, are more closely related than transit and driving.

We captured every real 21-mode score table used by destination choice: 30 batches and 107,854 rows. ActivitySim's dataframe reducer needed a median of 486.626 milliseconds for the complete captured sequence. The fused GPU kernel, including sending 18.119 MB to the GPU and returning the answers, needed 120.737 milliseconds in the longer 31-trial test: **4.030 times faster**. Even its slowest recorded GPU trial was faster than the fastest CPU trial. Its largest logsum difference was about 0.0000000000000036.

The full-model test alternated the Phase 6 and Phase 7 programs six times. Trip destination fell from 12.179 to 10.281 seconds, **1.185 times faster**. All 34 model steps fell from 54.459 to 52.166 seconds, **1.044 times faster**. Every Phase 7 run beat every Phase 6 run, and all seven final result files were byte-for-byte identical in all six trials.

Why is a 40x kernel only a 1.044x whole-model improvement? The kernel replaces less than half a second of CPU reduction. Most remaining time evaluates 404 behavioral expressions, samples destinations, runs other model components, and organizes data. This is Amdahl's law: speeding up a small slice cannot speed up the whole pie by the same amount.

Phase 8: current software and ten times more households

Earlier complete-model tests used 5,000 households. Phase 8 asked two harder questions. Does the integration still work with a pinned current ActivitySim development version? Does the advantage remain when the public workflow grows to 50,000 households across 190 zones?

The answer on this workstation is yes. The experiment alternated three regular ActivitySim runs with three ChoiceForge runs, always starting a fresh process:

Measured boundary	Regular ActivitySim median	ChoiceForge median	Speedup
Four scheduling components	30.1 s	23.7 s	**1.270x**
Trip destination	30.9 s	21.8 s	**1.417x**
All 34 model steps	147.225 s	131.812 s	**1.117x**

ChoiceForge saved a median of 15.413 seconds for the whole run. Every one of the three ChoiceForge runs was faster than every regular run. The seven final files matched byte for byte in all six trials. Those files describe 50,000 households, 111,130 people, 142,761 tours, and 350,751 trips.

The scale also shows what happens inside the computer. The largest scheduling call contains 9,561,750 feasible schedule rows for 50,325 choosers. Its compact input is 327.920 megabytes. The GPU part takes about 0.4 seconds, but preparing timetable facts, packing the data, retrieving controlled random numbers, and mapping the answer make the entire boundary about 3.3 seconds. This means the next scheduling improvement should focus on preparation and data reuse, not only on making the GPU arithmetic faster.

Phase 8 also captured a much larger nested-logit calculation: 30 real batches, 3,186,130 rows, and 535.270 megabytes of 64-bit mode scores. In 11 alternating trials, ActivitySim's CPU reducer needed a 4.646627-second median. The GPU, including both transfers, needed 0.125627 seconds: **36.988 times faster**. Every GPU trial beat every CPU trial, and the largest logsum difference was about 0.0000000000000053.

That 36.988x result is for one captured mathematical boundary. The 1.117x number is the honest complete-model result. The rest of ActivitySim still has to create people and tours, evaluate expressions, sample places, organize tables, and write outputs.

Phase 9: much larger geography and a useful stop sign

Phase 9 downloaded the public full-geography Prototype MTC data. The full data has 2,875,192 households, 7,566,527 people, and 1,454 zones. That is 7.7 times as many zones as Phase 8. A zone is a geographic area such as a neighborhood or traffic-analysis area, so more zones make the travel calculations much larger and more varied.

The whole population needs far more memory than this workstation has. The published configuration describes a 64-process, 432-gigabyte-RAM setup. The local test therefore used 50,000 households but kept all 1,454 zones. It ran one fresh regular ActivitySim process and one fresh ChoiceForge process.

For this test, ChoiceForge accelerated only trip destination. It left the schedule choices on the regular ActivitySim path. The final accessibility, household, participant, land-use, person, tour, and trip files were byte for byte identical. They describe 50,000 households, 132,536 people, 175,579 tours, and 442,682 trips.

Measured boundary	Regular ActivitySim	ChoiceForge	Observed result
Trip destination	39.2 s	28.0 s	**1.400x faster**
All 34 model steps	198.794 s	187.010 s	**1.063x faster**

This is one correctly matched pair, not yet a final statistical claim. A high-memory computer must run several alternating pairs before those numbers can be called reliable medians.

Phase 9 also found nine schedule choices that changed at a floating-point boundary when the experimental scheduling GPU path was enabled at 100,000 households. One later decision changed because the earlier difference affected the model's sequence of controlled random numbers. ChoiceForge therefore turns that scheduling path off for this configuration. This is not a failure to hide: it is exactly why the project checks complete output files rather than trusting a fast-looking timer. A faster answer that changes the model is not acceptable.

10. What the current evidence proves

The project can currently support these narrow statements:

- The fused linear-choice GPU kernel runs correctly on the tested NVIDIA hardware and software setup.
- It produced the same choices as the CPU reference in the reported synthetic benchmarks.
- Its logsums were numerically extremely close to the reference.
- For the reported large synthetic workload, it was much faster than the readable, materializing NumPy reference, even when transfers were included.
- For the scheduling-shaped 10,000-row workload, it was 4.19 times faster than a fused 24-core Numba CPU including transfers, with zero choice mismatches.
- The GPU advantage depends strongly on problem shape and size: it lost to the fused CPU on the 10,000-row, 32-alternative workload.
- Avoiding a full utility matrix can reduce intermediate memory use.
- ActivitySim-compatible random draws and final sampling rules can be preserved.
- Offloading sampling alone is not worthwhile when transfer time is included.
- Warm Sharrow production runtime and component shares are now measured reproducibly.
- Phase 2 replayed 4,477 real mandatory-tour choices with zero GPU mismatches.
- The real scheduling kernel is 8.57 times faster when inputs are GPU-resident, while the transfer-inclusive path is slower at 0.53 times CPU speed.
- Phase 3 reduces that input by 85.46 percent and makes the native transfer-inclusive GPU boundary 3.83 times faster than a generated 48-thread CPU implementation.
- Phase 4 makes the complete mandatory scheduling component 1.120 times faster than cached Sharrow in normal runs and 1.073 times faster in matched-checkpoint runs.
- Phase 5 makes the complete four-component scheduling family 1.272 times faster, led by a 2.836 times non-mandatory scheduling speedup.
- Seven substantive final CSV files are byte-identical across all Phase 5 trials, including all 9,806 tours and 23,583 trips.
- Phase 6 makes the packed real destination kernel 1.464 times faster than a batched Numba CPU including transfers, with zero mismatches across 3,971 choices.
- Phase 6 makes the complete trip-destination component 1.145 times faster and the complete 34-step prototype model 1.052 times faster in interleaved trials.
- Seven substantive final CSV files are byte-identical across all six Phase 6 A/B trials.
- Phase 7's transfer-inclusive nested-logit reducer is 4.030 times faster than ActivitySim's pandas reducer at the captured aggregate boundary, with maximum absolute error of 3.6e-15.
- Phase 7 makes trip destination 1.185 times faster and the complete example model 1.044 times faster than Phase 6; every optimized run beats every baseline run.

- Phase 16 makes the generated-utility trip-destination component 1.025 times faster in three repeated 50,000-household public pairs; every candidate destination time beats every baseline time and every modeled decision matches.
- Phase 17 strengthens that result to a 1.040-times destination speedup in five repeated pairs, with a positive 0.8-to-1.9-second bootstrap interval and zero changed modeled decisions.
- Phase 17's complete-model median is 1.006 times faster, but its bootstrap interval still includes a 0.042-second slowdown, so the strict whole-model gate remains failed.
- Seven substantive final CSV files are byte-identical across all six Phase 7 A/B trials.
- Phase 8's 3,186,130-row nested-logit replay is 36.988 times faster on the GPU including transfers, with maximum absolute error of 5.4e-15.
- On pinned current ActivitySim at 50,000 households, the four scheduling components are 1.270 times faster, trip destination is 1.417 times faster, and all 34 models are 1.117 times faster.
- All three Phase 8 optimized runs beat all three baselines, and seven substantive CSVs are byte-identical across all six runs, including all 142,761 tours and 350,751 trips.
- On public full MTC geography with 1,454 zones, Phase 9's destination-only GPU configuration has byte-identical outputs in one 50,000-household A/B pair and an observed 1.400x trip-destination ratio. It is a scale gate, not a median claim.
- The Phase 9 scheduling experiment found nine changed choices at 100,000 households, so that GPU path is disabled for full geography until a precision safeguard resolves them.

11. What the current evidence does not prove

It does **not** yet prove that:

- every ActivitySim model will run faster;
- every complete model will be faster;
- whole-model superiority for the Phase 17 generated-utility backend; its median is faster, but two nearly tied pairs and an interval including zero keep the strict gate from passing;
- a complete model will be 13.64 times faster;
- arbitrary ActivitySim utility expressions can run in the kernel;
- destination choice with thousands of irregular alternatives is solved;
- results will be identical on every GPU and software version; or
- a faster calculation makes the behavioral model more accurate.

The NumPy reference remains useful for correctness, and Phase 1 includes a strong fused 24-core CPU competitor. Phase 2 supplies the real scheduling replay, Phase 3 supplies transfer-inclusive scheduling-kernel superiority, Phase 4 supplies the first complete-component comparison, Phase 5 supplies breadth, Phase 6 supplies a destination replay, Phase 7 supplies batched setup plus a live nested-logit GPU boundary, Phase 8 supplies pinned-current 50,000-household evidence, and Phase 9 supplies a full-geography scale gate. A broader claim still requires a differently structured public model, other GPUs, high-memory repetition, and independent reproduction.

12. From promising prototype to convincing result

A responsible roadmap looks like this:

Step 1: Expand the supported math - completed for four scheduling components

Phase 3 compiles the real scheduling arithmetic and Boolean expressions directly from compact traveler and alternative columns. Phase 5 adds categorical assignments, inequalities, and different timetable owners. Stateful timetable functions remain precomputed primitives, and other ActivitySim components may require more operations.

Step 2: Build a configured ActivitySim backend - completed for the tour-scheduling family

Phase 4 adds an explicit setting, preserves ActivitySim-owned random draws and timetable updates, and falls back to ActivitySim for unsupported tracing or estimation paths. Phase 5 reuses that contract for four scheduling components.

Step 3: Handle large destination choices - completed for the prototype model

Phase 6 packs 30 real ragged destination segments into one launch and proves a transfer-inclusive GPU win. Phase 7 batches live preprocessing across purposes and adds the 21-mode nested-logit GPU reducer. Much larger regional alternative sets remain future work.

Step 4: Prove correctness on public benchmark data - completed for the prototype scheduling family

Phase 2 checks all six prototype MTC mandatory-scheduling batches and records every numerical tolerance. Phase 5 verifies byte-identical final files after four scheduling components. Phase 6 checks all 30 destination batches and hashes all six A/B model trials. Phase 8 repeats the proof at 50,000 households on current ActivitySim. Phase 9 repeats the destination test across 1,454 public MTC zones and disables the scheduling path when its exactness gate fails. A differently structured public model is still needed before making a general claim.

Step 5: Run fair performance comparisons - completed for four scheduling components

Phase 5 compares three cached-Sharrow and three transfer-inclusive ChoiceForge scheduling runs. Phase 6 strengthens the design with alternating A/B processes and proves both component and whole-model gains. Other models and hardware still need the same treatment.

Step 6: Publish a reproducible evidence package

Include exact commands, pinned dependencies, raw timing samples, correctness reports, benchmark data instructions, and results from more than one GPU. Let other researchers reproduce or challenge the claim.

13. Why faster travel modeling could matter

Faster computation does not automatically make a better transportation decision. But it can change what analysts have time to study.

More scenarios

Instead of testing only a few projects, an agency could examine more combinations of fares, services, land use, road pricing, and policy assumptions.

Better uncertainty analysis

No forecast is certain. Faster models can support many runs with different population growth, fuel prices, remote-work rates, or random seeds. The range of results may be more informative than one forecast.

Faster feedback

Modelers could find data or configuration problems sooner, shorten development cycles, and respond more quickly to community questions.

Lower intermediate memory pressure

Fused calculations that avoid giant temporary tables may allow larger problems or reduce memory bottlenecks.

Important cautions

Speed cannot repair biased survey data, missing travel options, unrealistic assumptions, or unfair project goals. GPUs use electricity and require specialized hardware and skills. Public decisions still need transparency, equity analysis, local knowledge, and human judgment.

14. Frequently asked questions

Does the GPU predict travel differently?

It should implement the same model rules. The goal is to calculate them faster, not invent a different behavior model.

Does zero choice mismatch mean every number is identical?

Not always. Some kernel-level logsums differ by tiny amounts because floating-point arithmetic can be ordered differently in parallel, even when the selected alternative is unchanged. In Phases 5, 6, and 8, however, all seven substantive final CSV files were byte-for-byte identical in the reported A/B trials.

Why use 32-bit numbers?

They use half the memory of 64-bit numbers and are usually faster on GPUs. Whether they are accurate enough must be tested for each model. ActivitySim-compatible final sampling also has a 64-bit path in the adapter.

Why not move the entire model to the GPU immediately?

ActivitySim contains varied expressions, tables, joins, tracing, estimation tools, and irregular choice sets. A small, well-tested component creates a safer foundation than a large rewrite whose errors are hard to locate.

Is a GPU always faster?

No. Small workloads, frequent transfers, branching logic, or underused cores can make a GPU slower. The isolated sampling benchmark demonstrated this directly.

Which speedup is the headline?

Use the number for the boundary being discussed. Phase 3's scheduling kernel is 3.83x, Phase 7's smaller nested-logit reducer is 4.030x, and Phase 8's much larger nested-logit replay is 36.988x. At the 50,000-household scale, Phase 8 makes the four complete scheduling components 1.270x faster, complete trip destination 1.417x faster, and the complete model 1.117x faster. Phase 9's 1.400x destination ratio uses much larger 1,454-zone geography, but is only one matched pair. Only a carefully repeated full-model number should be called a whole-model speedup.

15. Mini glossary

Term	Plain-English meaning
ActivitySim	An open-source framework for activity-based travel demand models.
Amdahl's law	The idea that total speedup is limited by parts of a job that were not accelerated.
Alternative	One possible choice, such as walk, bus, or car.
Availability	A rule saying whether an alternative is possible.
Benchmark	A controlled experiment that measures speed and correctness.
Chooser	The person, household, tour, or trip making a simulated decision.
Compiler	A program that translates checked instructions into another executable form.
Compact representation	Data stored without unnecessarily repeating the same facts.
CPU	A flexible general-purpose processor with a modest number of powerful cores.
CUDA	NVIDIA's platform for programming its GPUs.
Feature	An input fact, such as travel time, cost, income, or vehicle access.
Floating point	The computer's approximate way of storing non-whole numbers.
Fusion	Combining connected calculations so intermediate data is not repeatedly stored or moved.
GPU	A processor with many simpler cores suited to large parallel calculations.
GPU kernel	A small program executed by many GPU threads.
Logsum	A summary of the attractiveness of all available alternatives.
Nested logit	A choice calculation that groups related alternatives before combining them.
Probability	A number from 0 to 1 describing how likely an outcome is.
Shared memory	Small, fast GPU memory close to threads in one block.
Skim	A table of travel time, distance, cost, or related values between places.
Synthetic population	Anonymous simulated households and people resembling a real region.
Utility	A model score representing how attractive an alternative is.

16. The bottom line

ChoiceForge asks a focused question: can a common bundle of travel-choice calculations be redesigned as one correctness-first GPU kernel that avoids huge intermediate tables?

The answer is now more precise. The fused kernel can decisively beat a strong 24-core CPU when each chooser has enough alternatives and features, but it loses on small, narrow jobs. Phase 1 also found a multi-warp synchronization defect and two near-boundary choice differences, demonstrating why wider correctness tests and precision rules matter as much as speed.

Phases 3 through 9 advanced integration through full MTC geography. Phase 8 improved whole-model time with exact outputs. Phase 9 confirmed exact destination results across a far larger zone system and held scheduling back until precision improves. Phase 11 then repeated that full-geography destination test and made the outcome stronger. Phase 12 identified the utility-calculation opportunity. Phase 13 completed the exact CPU answer key. Phase 14 generated the

matching GPU evaluator and proved exact agreement on real public-model batches, while keeping it safely in shadow mode.

17. The latest evidence: replicated destination speedup and a safer path to GPU utilities

This section updates the earlier phases. It uses two labels deliberately:

- **Proven production result** means a complete public ActivitySim run finished, its final files matched exactly, and the timing was repeated.
- **Foundation or microbenchmark** means a smaller, controlled engineering test succeeded. It is useful evidence, but it is not yet a claim that a whole ActivitySim model got faster.

Phase 10: a guardrail, not a shortcut

The full-geography scheduling experiment revealed a subtle risk. A GPU and CPU can both follow the same-looking formula yet round a last digit differently. If a random draw lands exactly near the boundary between two choices, that last digit can change the selected alternative. One changed early choice can also change the controlled sequence of later random draws.

Phase 10 added a **shadow guard**. Think of it as a second answer key running beside an experimental route. The established CPU/Sharrow result remains authoritative. The GPU is allowed to report success only if its shadow comparison passes; otherwise the system deliberately returns the CPU result. This is slower than blindly using the GPU, but it prevents a silent model change while the arithmetic issue is investigated.

Phase 11: the current headline result

Phase 11 focused on the part that was already exact: trip destination. It used the public Prototype MTC full geography with 1,454 zones and 50,000 households. Three fresh, interleaved A/B pairs were run: regular ActivitySim, then ChoiceForge, with the order alternated to reduce the chance that unrelated computer activity picked the winner.

Measured boundary	Regular ActivitySim median	ChoiceForge median	Result
Trip destination	39.7 s	28.6 s	**1.388x faster**
All 34 model steps	202.492 s	190.380 s	**1.064x faster**

The complete run saved a median of 12.112 seconds. The three paired whole-model savings were 12.172, 12.112, and 7.596 seconds. A simple resampling check placed the median saving between 7.596 and 12.172 seconds for these three pairs. Every optimized run was faster than its paired baseline.

Most importantly, all six substantive final CSV output files were **byte-for-byte identical** in every pair. That is stronger than saying the choices merely looked similar: the saved files matched character for character. The experiment also pins the exact ActivitySim source revision, configurations, patches, and hashes, so another person can repeat the same setup.

This is the honest current headline: on this workstation, for this public 50,000-household full-geography workload, ChoiceForge made the complete model 1.064 times faster while reproducing its reported final outputs exactly. It does not mean every ActivitySim model, computer, or future version will gain the same amount.

Why the next bottleneck is utility calculation

Destination choice works in two broad stages. First, the model evaluates many utility equations: for each potential destination it combines travel time, cost, household facts, destination facts, and rules. Second, it reduces those scores into probabilities and a selected destination.

ChoiceForge already makes the second stage faster. But the Phase 11 profiler showed that preparing and sending the first stage's finished utility values involved about 12,564,936 rows, or roughly 2.111 gigabytes of data. Sending a huge finished answer sheet to the GPU is like carrying every completed math problem to a very fast calculator. The more ambitious improvement is to send the compact ingredients and calculate the utility scores on the GPU too.

Phase 12: proving the ingredients can be translated safely

ActivitySim utility equations are written as expressions such as "travel time times a coefficient, plus an income term, only when a condition is true." A GPU cannot safely accelerate these expressions merely by guessing what they mean. The order of arithmetic, special functions, missing values, and table lookups all matter.

Phase 12 therefore built three safety layers:

1. An **expression reader** turns the public trip-mode-choice formulas into a small checked tree of operations. It successfully parses and evaluates all 253 distinct expressions used in that configuration on both CPU and GPU test paths.
2. A **skim adapter** gives the GPU path access to travel-time and cost lookup tables in the same way the model asks for them. Its tests confirm the adapter returns the expected values.
3. A **shadow capture** runs the GPU calculation beside the trusted CPU/Sharrow calculation without changing the official model answer. It records any difference for investigation.

The good news is that the deterministic GPU kernel exactly matched the local ordered CPU expression evaluator in the shadow test. The caution is that a compiled path can use different input precision, operation grouping, reduction order, or optional speed transformations such as `fastmath`. Any of these can change rounding in the last few binary digits. The shadows found differences from tiny fractions up to 0.25 for extremely large "unavailable" scores. No GPU utility result is allowed to replace Sharrow's production result until one explicit cross-device policy controls both targets.

A controlled speed test: encouraging, but not a production claim

Phase 12 also tested a deliberately simple 250,000-row, 64-feature, 21-alternative utility-and-choice pipeline. The CPU NumPy reference took 235.215 milliseconds. The GPU lowering plus 21-mode reduction took 28.988 milliseconds, an **8.114x** pipeline speedup, while the logsum check stayed within 0.00000000001.

This result answers a narrow question: can the compact-ingredient design be fast enough to matter? Yes. It does **not** yet prove an 8.114x whole-model improvement, because the test is not the full ActivitySim destination component and the CPU reference is not Sharrow's compiled production evaluator.

The strict IR plan: one recipe, two kitchens

The project is now pursuing a more rigorous solution rather than trying to tune away discrepancies one at a time. A **strict intermediate representation**, or strict IR, is a precise shared recipe for an equation. Instead of separately interpreting an expression for the CPU and generating a similar-looking CUDA kernel for the GPU, both targets are built from the same checked list of operations and numeric rules.

An everyday analogy: two kitchens can make the same named dish but use different measuring cups and a different order of steps. A strict recipe states the ingredients, measurements, order, and rounding rules so both kitchens produce the same dish. For ChoiceForge, the two kitchens are the strict CPU evaluator and the generated CUDA target.

The strict IR has generated a canonical description of the public MTC trip-mode utility: 379 terms across 21 alternatives. Phase 13 completed the strict CPU target, including separate ordered multiply and add steps, exact comparison reports, and fail-closed policy checks. Phase 14 completed the CUDA target generated from the same IR. The project test suite now passes 95 tests, including exact cross-device edge cases, compact skim gathering, a device-resident handoff to the nested-logsum reducer, the Phase 16 FP32 arithmetic policy, Phase 17 plan reuse, and Phase 18 GPU-native runtime checks.

This remains a project implementation, not a completed upstream Sharrow feature. Phase 15 removed the qualification-only utility transfer but failed its 50,000-household scale gate. Phase 16 recovered a repeated large destination-component win. Phase 17 added persistent plans and trip-mode continuation, strengthening that component win and making the five-run whole-model median faster. The strict path remains opt-in because the whole-model interval still includes zero and replication on other hardware and models is unfinished.

How to read the project status today

Question	Best current answer
Is there a real, repeated, exact speedup?	Yes. Phase 11 achieved 1.064x for the complete 50k-household, 1,454-zone public run, with byte-identical outputs in three pairs.
Is the destination component itself faster?	Yes. Its Phase 11 median was 1.388x faster.
Has the large utility equation been connected to the real GPU path?	Yes. Phase 17 reuses generated plans for destination and trip-mode utilities, with Sharrow fallback and explicit telemetry.
Is the compact GPU utility approach promising?	Yes. It was 8.114x faster in a controlled microbenchmark with a tight numerical gate.
What prevents a bigger claim today?	The destination component wins, but whole-model timing noise still crosses zero; a second GPU and a second public model have not been tested.
What changed in Phase 16?	An explicit FP32 policy, compact inputs, and caching produce a repeated 1.025x large destination-component win. The whole-model gate still fails.
What changed in Phase 17?	Checked persistent plans and trip-mode continuation produce a five-pair 1.040x destination win and a 1.006x faster whole-model median; the strict whole-model gate still fails.

18. Phase 13: building the exact CPU answer key

Phase 12 discovered that saying "use the same formula" was not precise enough. Two compilers may change when a number is rounded or combine a multiplication and addition. The answers usually remain close, but a travel model can place a random draw very near a choice boundary. That makes the last few digits important.

Phase 13 turns the arithmetic rules into a written and executable contract. The strict CPU evaluator is like an official answer key. It requires:

- 64-bit arithmetic while an expression is being calculated;
- one conversion of the finished feature to a 32-bit number;
- 32-bit coefficients and utility totals;
- the original term order;
- a separate multiplication and addition for every term; and
- normal IEEE rounding, with speed shortcuts disabled.

If the IR version, policy, or identifying hash changes unexpectedly, the evaluator refuses to run. It also refuses unresolved coefficients and malformed arrays. These are useful failures: they stop a plausible-looking but undefined calculation from entering the model.

Did it cover the real model?

Yes. The canonical public MTC utility contains 379 terms for 21 travel-mode alternatives. Every term and alternative executes under the strict policy. An independent, simple scalar loop produced exactly the same utility bits. The complete repository now passes 95 tests.

Phase 13 then ran the public full-geography model with 1,001 households. It observed 30 real trip-mode batches containing 85,126 rows. That meant comparing 32,262,754 individual feature values and 1,787,646 utility values. ActivitySim completed all 34 model steps normally in 95.511 seconds, and Sharrow remained the official source of every model answer.

What did the comparison teach us?

The new strict answer key and current Sharrow did not match exactly. They agreed on 99.9118 percent of feature cells and 66.8085 percent of utility cells. The largest feature difference was about 0.0000305. The largest utility difference was 0.25, occurring among very large scores where a float32 number has relatively wide spacing.

Those percentages do not mean Sharrow is "66.8 percent correct." Sharrow follows its existing compiled arithmetic, while the strict evaluator follows the newly published cross-device policy. Many different utility cells can result from one tiny feature difference or a different order of hundreds of additions. Phase 13's job is to expose and classify that fact, not silently declare one existing compiler's behavior universal.

Every observed difference now has a stage. The reports found 28,466 feature cells whose compiled-expression result did not follow the strict expression policy. Among 593,346 different utility cells, 183 were explained by those input differences alone, while 593,163 also reflected a different accumulation rule. The first feature difference was about 0.00000763; the first utility difference was one float32 step.

Why Phase 13 is successful even though Sharrow differs

The goal was to create one stable answer key for future CPU and GPU code, not to copy whatever rounding happens on this workstation. All 379 terms and 21 alternatives run under that answer key. The comparison gate identifies the first different row, expression, alternative, values, and arithmetic stage. It can operate in observation mode, where Sharrow remains authoritative, or exact mode, where any difference stops qualification.

Phase 14 has now generated CUDA from this same strict IR and matched the strict CPU arrays exactly. Current Sharrow remains the safe ActivitySim fallback. The project will attempt another complete-model performance claim only after the generated path is integrated without qualification overhead and passes repeated byte-identical trials.

19. Phase 14: giving the GPU the exact same recipe

Phase 13 made the official answer key. Phase 14 built a code generator that turns that same checked recipe into a CUDA program. This matters because two separately handwritten programs can look equivalent while making tiny different choices about types, rounding, or operation order. Generating both evaluators from one hashed recipe greatly reduces that ambiguity.

What does the generated GPU program do?

For each model row, one GPU work group evaluates all 379 travel terms. It stores those temporary feature values in fast shared memory. Then separate GPU workers calculate the 21 travel-mode scores. Each worker uses the coefficients in the original order and performs a separate 32-bit multiplication and addition for every term. The resulting 21 scores can remain on the GPU and go directly into the nested-logsum calculation instead of taking a round trip through ordinary computer memory.

Inputs are packed by their real meaning: decimal numbers, whole numbers, and true/false values are not silently mixed. The compiled program is cached using the recipe hash, input types, coefficients, generated source, and compiler rules. Change any important ingredient and ChoiceForge builds a different kernel instead of reusing a stale one.

A tiny-number rule discovered by testing

Phase 14 found a useful edge case. A 32-bit floating-point number can be so tiny that it enters a special range called **subnormal**. The NVIDIA runtime on this machine turns such numbers into signed zero during these calculations. This is called **flush to zero**.

Ignoring the behavior would leave the phrase "exactly the same arithmetic" with a hidden exception. Instead, strict IR version 3 publishes the rule. The CPU answer key now also turns float32 subnormals into signed zero after the same steps. Tests deliberately use subnormal numbers, NaN ("not a number"), and infinity. This makes the cross-device agreement a real contract rather than an agreement that only holds for convenient inputs.

Did it match on the public model?

Yes. The same public 1,001-household, full-geography ActivitySim workload produced 30 trip-mode batches and 85,126 rows:

Exact Phase 14 check	Result
Batches	30 of 30
Feature values	32,262,754 of 32,262,754
Utility values	1,787,646 of 1,787,646
Largest feature difference	0.0
Largest utility difference	0.0
Terms and alternatives per batch	379 and 21

Every checked GPU bit matched the strict CPU answer key. ActivitySim still used Sharrow's established output as the official answer, so the experiment could not alter model behavior. That is the safe way to qualify a new backend.

Is it faster?

The generated kernel's diagnostic median was about 5.868 milliseconds. Preparing and transferring its inputs took about 116.157 milliseconds, and downloading the large comparison results took about 1.593 milliseconds. These numbers are not a fair speed contest. Phase 14 intentionally captures every one of the 379 feature columns so it can prove exactness term by term, while production code would keep compact inputs and useful outputs on the GPU.

Therefore the honest result is: **Phase 14 proves exact generated CPU/GPU semantics, not a new end-to-end speedup.** A separate test already confirms that the generated 21-column utility matrix can stay on the GPU through the nested-logsum reducer with no utility download and no reducer re-upload.

What does this change?

Before Phase 14, different arithmetic rules blocked the more ambitious GPU utility path. That obstacle is now resolved for this IR and hardware stack. That was the remaining Phase 15 problem: remove shadow-only transfers, connect generated utilities to the real path, and repeat public-model A/B runs. The next section explains what happened when those tests were completed.

20. Phase 15: putting the exact GPU recipe into the real model

Phase 14 proved that the CPU and GPU could follow the same recipe. Phase 15 asked a harder question: can the GPU's answer actually replace Sharrow's answer inside ActivitySim, stay on the graphics card for the next calculation, and make the real model faster?

The first connection worked, but exposed a data problem

The first candidate made all 379 ingredients as ordinary columns before the GPU kernel ran. Imagine copying every number from a library of road-time maps onto one enormous worksheet, sending the worksheet to a calculator, and then throwing it away. The arithmetic was fast, but preparing the worksheet was not.

At 1,001 households, the first repeated combined test looked faster than plain ActivitySim. A fairer test then compared Phase 15 directly with Phase 11, so both sides received the same destination batching and GPU nested-logsum work.

That test showed the first strict utility path itself was slower. Measuring the parts found the cause: looking up and materializing skim columns took about 15.35 seconds across three candidate runs, while the useful GPU work took much less time.

The compact skim design

ChoiceForge was changed so the generated kernel receives:

- the shared road-time and cost map cubes;
- the already-checked origin, destination, and time positions for each row;
- the smaller ordinary chooser columns; and
- the strict coefficients and recipe.

The kernel now looks up a needed map value directly. It does not build a giant 379-column feature table. Identical map cubes are uploaded only once, even when ActivitySim creates new temporary wrapper objects. After trip destination is finished, the cache is released so later model steps do not pay for unused GPU memory. If any part fails, ChoiceForge uses Sharrow instead.

Did compact GPU gathering stay exact?

Yes. The final qualification covered 30 real public-model batches:

Check	Exact result
Rows	85,126
Feature values	32,262,754 of 32,262,754
Utility values	1,787,646 of 1,787,646
Largest strict CPU/GPU difference	0.0
Utility downloads before nested logsum	0 bytes
Nested-logsum utility re-uploads	0 bytes

Every modeled trip decision also matched. A printed diagnostic called `destination_logsum` differed by at most 0.000008 at this scale. That field is not used as a later decision in this output check. The difference comes from the published strict arithmetic versus current Sharrow arithmetic; the strict CPU and GPU answers still match bit for bit.

The direct repeated speed result

Three fresh pairs compared Phase 11 with compact Phase 15 at 1,001 households. The destination component improved from a median 11.3 seconds to 10.3 seconds, or **1.097 times faster**. Every Phase 15 destination run was faster than every Phase 11 destination run. All 90 strict utility batches stayed on the GPU for the nested calculation, and all modeled decisions matched.

The complete model did not pass the stricter promotion rule. Its median was 84.631 seconds for Phase 11 and 85.445 seconds for Phase 15. Other small model steps varied enough to hide the one-second destination gain. ChoiceForge therefore claims component superiority at this qualification scale, not a new whole-model record.

What happened at the large scale?

The final diagnostic used 50,000 households, all 1,454 zones, and 4,188,312 utility rows. Decisions still matched, but compact Phase 15 took 33.3 seconds for trip destination versus 28.4 seconds for Phase 11. The complete model took 197.871 versus 192.519 seconds. This is a clear rejection, so running three more expensive pairs would not create a responsible speed claim.

The reason is now narrower and useful. The GPU kernel gives one block to each row and makes many scattered reads from large skim cubes. At 50,000 households, that access pattern is slower than Sharrow's compiled expression path. The next compiler should process tiles of nearby rows, combine repeated map reads, reuse a gathered value across multiple terms, and keep the exact ordered arithmetic rules.

What should a high-school reader conclude?

An experiment can succeed even when the final answer is "do not deploy this version." Phase 15 produced working, exact, fail-safe GPU integration and a repeated component win. It also used a public large benchmark to prove where the design stops winning. The production headline therefore remains Phase 11's repeated 50,000-household result. Phase 15 is the tested bridge to a better future compiler, not an excuse to replace a faster working system today.

21. How the complete calculation fits together

The earlier sections introduce each idea separately. This is the whole trip destination calculation in one view:

```
public households, people, zones, and travel-time maps
                          |
                          v
ActivitySim creates rows representing possible trip destinations
                          |
                          v
utility equations score 21 travel modes for each row
                          |
                          v
nested logsum summarizes how useful those modes are
                          |
                          v
destination probabilities are formed and one destination is selected
                          |
                          v
the selected trip and diagnostic values are written to final output files
```

Phase 11 accelerates the preparation and nested-logsum part of this pipeline. Phase 15 additionally tries to calculate the 379 utility terms on the GPU and hand their 21 results directly to the GPU nested-logsum calculation. Keeping that handoff on the GPU avoids downloading a large utility table to ordinary memory and immediately uploading it again.

What exactly is a skim cube?

A **skim** is a lookup table containing travel time, distance, cost, or another measure between two zones. Imagine a map grid: choose an origin row and a destination column, and the cell tells you the travel time. A model needs many such maps for different travel modes and times of day. Stacking the maps makes a three-dimensional **skim cube**.

For every possible trip, the kernel receives the already-checked origin, destination, and time positions. It uses them like coordinates to read the needed cells. At small scale this avoids building a giant worksheet and is faster. At large scale, millions of rows may ask for far-apart cells. Those scattered reads are like repeatedly fetching books from unrelated library shelves: the arithmetic workers spend too much time waiting for data.

The next design should group nearby requests into **tiles**, reuse a fetched skim value when several utility terms need it, and arrange memory reads so neighboring GPU workers fetch neighboring data. This is a data-movement problem, not evidence that the equations are too difficult for a GPU.

22. Three different meanings of "the answers match"

The word **exact** can describe different boundaries. A responsible report must say which boundary it tested.

Level	Plain-English question	Phase 15 result
Arithmetic	Did the strict CPU and generated GPU follow the same numeric recipe bit for bit?	Yes: all 32,262,754 checked feature cells and 1,787,646 utility cells matched exactly.
Decisions	Did the model choose the same alternatives?	Yes: every modeled trip decision matched in the reported runs.
Saved files	Were complete output files identical byte for byte?	All six non-trip substantive files were byte-identical. Every modeled field in `final_trips.csv` matched. The reported `destination_logsum` diagnostic was bounded, not byte-identical.

The small `destination_logsum` difference is between current Sharrow arithmetic and the newly published strict arithmetic. It was at most 0.000008 for 1,001 households and 0.00001 for 50,000 households, below the 0.0001 gate. It is not a disagreement between the strict CPU and GPU, which matched bit for bit.

This distinction also explains why Phase 11 has the strongest production replication statement. In its repeated 50,000-household trials, all substantive final CSV files were byte-identical. Phase 15 has exact strict cross-device arithmetic and exact decisions, but one diagnostic column intentionally exposes the known difference from Sharrow's arithmetic order.

23. The rule for promoting an experiment

One fast run can be luck. The computer may be warmer, another program may have used a processor core, or a file may already be cached. ChoiceForge therefore uses fresh processes and alternates conditions: A1, B1, A2, B2, A3, B3. A is the established implementation and B is the candidate. The **median** is the middle of three timings, so one unusually fast or slow run has less influence.

For automatic promotion, all of these conditions must pass:

1. Run at least three fresh, interleaved A/B pairs.
2. Preserve every modeled decision and pass the complete-output correctness gate.
3. Make the median trip-destination time faster.
4. Make the median complete-model time faster.
5. Achieve complete separation: every candidate time must beat every corresponding baseline time.

Phase 15 qualification gate at 1,001 households	Result
Three fresh interleaved pairs	Pass
Exact modeled decisions	Pass
Faster destination median	Pass: 11.3 to 10.3 seconds
Every candidate destination run faster	Pass
Faster complete-model median	Fail: 84.631 to 85.445 seconds
Every candidate complete-model run faster	Fail

Phase 15 qualification gate at 1,001 households	Result
Final promotion	**Rejected**

The 50,000-household test used one pair. One pair cannot prove superiority, but it can reveal that a candidate is clearly unpromising: Phase 15 was slower at both the destination and complete-model boundaries. Repeating that losing configuration three more times would spend hours without fixing its identified memory-access problem. A redesigned kernel must return to the full three-pair gate before any new speed claim is made.

24. Hardware, memory, and the boundary of the claim

The numbers in this guide describe one measured system, not every computer.

Reproduction item	Recorded value
CPU	AMD Ryzen Threadripper PRO 5965WX, 24 cores and 48 threads
System memory	63.9 GB usable for the benchmark workstation
GPU	NVIDIA RTX A4000, 16 GB, compute capability 8.6
NVIDIA driver	571.59
Python	3.11.14, 64-bit
ActivitySim source	Commit `16ab11180a26912987eb902daf945e268f3efc11`
Public geography	Prototype MTC Extended, 1,454 zones

The 50,000-household Phase 15 candidate reached 9,196 MiB of GPU memory. It did not run out of the A4000's 16 GB. The observed slowdown instead points to the one-block-per-row design and scattered skim reads. Another GPU with different memory bandwidth, cache sizes, or scheduling could behave differently, so the claim is deliberately limited to this hardware and software stack until other systems reproduce it.

The full public population contains 2,875,192 households. The upstream multi-process instructions call for hundreds of gigabytes of system memory, far beyond this workstation. A 50,000-household sample with all 1,454 zones is therefore the local public scale gate. It is large enough to expose the failed Phase 15 access pattern, but it is not a substitute for a full-population run on a high-memory server.

25. A practical reproduction checklist

Reproduction means more than downloading the code and seeing a program finish. Another researcher should be able to identify the same source, data, configuration, hardware, commands, outputs, and decision rule.

Pinned evidence

Item	Identifier recorded by the run
ChoiceForge Phase 15 evidence commit	`7f40ce227e46a16801acd6abc264544457c97a7b`
ActivitySim commit	`16ab11180a26912987eb902daf945e268f3efc11`
Prototype MTC `data_full.tar.zst` SHA-256	`b402506a61055e2d38621416dd9a5c7e3cf7517c0a9ae5869f6d760c03284ef3`
ActivitySim integration patch SHA-256	`aa2eed95d99fe4853365b9ed49427722ac467ef928013ccf7944fd9eda0e04e2`
Python environment lock SHA-256	`84b97738100cd4b8c405af50ff823cdddf4a33f5f203deab7839e4a8739a3adc`

Set up and verify the software

The detailed setup is in `docs/activitysim-integration.md`. In PowerShell, the important verification steps are:

```
git clone https://github.com/ActivitySim/activitysim.git tmp\activitysim-phase8-source
git -C tmp\activitysim-phase8-source checkout 16ab11180a26912987eb902daf945e268f3efc11
git -C tmp\activitysim-phase8-source apply ..\..\integration\activitysim-current-choiceforge.patch
uv venv --python 3.11 .venv-phase8
uv pip install --python .venv-phase8\Scripts\python.exe -e tmp\activitysim-phase8-source -e ".[gpu,
test]"
.\.venv-phase8\Scripts\python.exe -m pytest -q
```

Place the public full-geography data at `benchmark-data/phase9-mtc-full/prototype_mtc_extended/data_full`. Check its downloaded archive against the SHA-256 value above before extracting it. The benchmark scripts refuse to overwrite an existing run, so use a new short `RunTag` when repeating an experiment.

Run the exactness and performance gates

First run one qualification candidate. Then run three direct Phase 11 versus Phase 15 pairs:

```
.\scripts\run_phase15_candidate.ps1 -Households 1001 -RunTag reproduce-gate -MaxCandidateRows 20000
00
.\scripts\run_phase15_incremental_ab.ps1 -Households 1001 -Repetitions 3 -RunTag reproduce-r3 -MaxC
andidateRows 2000000
```

The candidate is explicitly enabled with `CHOICEFORGE_STRICT_CUDA_CANDIDATE=1`; it is off by default. The row policy `CHOICEFORGE_STRICT_CUDA_MAX_ROWS` can reject an ineligible batch before a large allocation. Any code-generation or GPU-reduction failure falls back to Sharrow, which remains authoritative.

Compare a new run with these machine-readable evidence files:

- `benchmark-results/phase15-candidate-summary.json`
- `benchmark-results/phase15-p15finalr3-summary.json`
- `benchmark-results/phase15-p15compact50-summary.json`

The manifests record source, patch, environment, data and configuration hashes, GPU and driver, Python version, individual timing samples, memory measurements, correctness results, and the final promotion decision. Matching the published median alone is not enough; the correctness and promotion fields must also pass.

26. The Phase 16 plan that was tested

Phase 15 showed that the next major opportunity was not to add more arithmetic to the existing one-block-per-row kernel. The approved Phase 16 plan was to change how data reaches that arithmetic and test each idea honestly:

1. Add measurement for skim-cache hits, memory transactions, and time spent waiting on gathers.
2. Sort or tile rows by origin, destination, and time without changing their final model order.
3. Coalesce neighboring skim reads and reuse one gathered value across every term that needs it.
4. Fuse tiled gathering, strict ordered utility accumulation, and nested-logsum reduction without writing a global feature or utility matrix.
5. Preserve strict IR version 3, typed inputs, default-off activation, bounded diagnostics, and Sharrow fallback.
6. Pass unit tests and exact CPU/GPU edge cases before running ActivitySim.
7. Qualify all 30 public-model batches and exact final decisions at 1,001 households.
8. Require three interleaved 50,000-household pairs and the full promotion rule before replacing Phase 11.
9. Repeat on at least one different GPU and one differently structured public ActivitySim model.
10. Only then propose the backend for upstream Sharrow or ActivitySim integration; until every gate passes, Phase 11 remains supported and Phase 15 remains opt-in research evidence.

27. Phase 16: the GPU finally wins the large target component

Phase 15 taught us that moving a calculation to a GPU does not automatically make it faster. Phase 16 used that failure as a map. It measured where time was going, tried several designs, rejected the slow ones, and found one policy that was both reproducible and faster for the targeted part of the public model.

Four ideas in ordinary language

First, constants no longer become giant repeated columns. If every row uses the same number, the GPU receives that number once. This is like writing a classroom rule on the board instead of printing the same rule on every student's paper.

Second, two names that point to the exact same column share one stored copy. Third, travel-time maps and compiled equation plans are cached so the program does not rediscover identical information for every batch. Fourth, an optional policy uses 32-bit floating-point arithmetic for the expressions instead of 64-bit arithmetic.

The last change matters on this NVIDIA RTX A4000. It can do ordinary 32-bit GPU arithmetic much faster than 64-bit arithmetic. Sharrow already stores each completed feature as a 32-bit number before combining features with coefficients, so Phase 16 publishes a separate 32-bit recipe and a separate CPU answer key for it. The older strict 64-bit recipe is still available and is not silently changed.

What is a numeric policy?

A **numeric policy** is a written recipe for how the computer handles numbers. It says how many bits a number uses, when rounding happens, whether operations may be rearranged, and how unusual values such as infinity are treated. Two programs can use the same equation but get slightly different last digits if their numeric policies differ.

ChoiceForge now has two named policies:

Policy	Expression arithmetic	Why it exists
Strict IR version 3	64-bit, then one 32-bit feature rounding	strongest continuation of the Phase 13-15 arithmetic contract
Phase 16 FP32	32-bit expression arithmetic and 32-bit feature storage	mirrors the deployed intermediate precision more closely and runs efficiently on this GPU

The run manifest records which policy was selected. The generated source hash also changes, so evidence from the two policies cannot be accidentally mixed.

Did the new CPU and GPU answers match?

Yes. On the real 1,001-household qualification:

Exact qualification question	Result
Real batches checked	30 of 30
Rows checked	85,126
Feature cells exactly equal	32,262,754 of 32,262,754
Utility cells exactly equal	1,787,646 of 1,787,646
Largest CPU/GPU difference	0.0
Fallback batches	0

The tests also include a simple equation where the 64-bit answer is 1 and the 32-bit answer is 0 because a very large 32-bit number cannot represent a change of just 1. That test proves the two policies really are different. It then proves that the published 32-bit CPU and GPU recipes agree exactly.

The large repeated performance proof

The public scale gate uses 50,000 households, all 1,454 zones, and 4,188,312 utility rows. It launches fresh processes in the order baseline 1, candidate 1, baseline 2, candidate 2, baseline 3, candidate 3.

Trip destination	Run 1	Run 2	Run 3	Median
Phase 11 baseline	28.5 s	28.5 s	28.8 s	28.5 s
Phase 16 FP32 GPU	28.4 s	27.7 s	27.8 s	27.8 s

Every candidate time beat every baseline time. The median speedup is 1.025 times, and the paired savings are 0.1, 0.8, and 1.0 seconds. Every modeled trip decision matched in all three candidate runs. The largest printed diagnostic logsum difference was 0.000010, below the 0.0001 limit.

This means the **component promotion gate passes**. It is a practical, repeatable GPU win for the part of the model Phase 16 was designed to replace.

Why is the whole model not promoted?

The full model contains many steps Phase 16 does not change. Their timings move slightly from run to run because of operating-system scheduling, file caches, CPU activity, and other noise. Whole-model medians were:

Complete model	Median
Phase 11 baseline	193.014 s
Phase 16 candidate	194.312 s

That is a 0.993-times result, which is slower, not faster. One candidate run was faster than its paired baseline, but two were slower. The **whole-model gate fails**. Reporting both gates prevents a real kernel success from being hidden and prevents that success from being exaggerated into an application-wide claim.

Slow ideas that were tested and rejected

Phase 16 did not simply keep the first plausible code:

- Loading 149 skim maps cooperatively for tiles of nearby rows was exact but synchronization and unnecessary eager loads made it slower.
- The model has only about 419-465 nonzero coefficients out of 7,959 positions. Skipping zeros sounds ideal, but one version created bulky control flow and another caused scattered shared-memory reads. Both were slower.
- Strict FP64 compaction reduced host packing from about 2.4 to 1.1 seconds and utility time from about 4.5 to 4.1 seconds, but destination still lost 32.5 to 30.1 seconds at large scale.
- FP32 reduced the large generated utility work to roughly 1.2-1.6 seconds and produced the repeated component win.

The lesson is that counting arithmetic operations is not enough. GPUs care about memory order, groups of threads following the same instruction, code size, and the kinds of number hardware they execute efficiently.

What Phase 17 needed to do

Phase 17 had three jobs: stop rebuilding the same GPU program, prevent that optimization from moving a delayed Sharrow cost into trip mode choice, and repeat the public benchmark enough times to separate a real destination win from ordinary timing noise. The next section explains what was implemented and what the measurements do and do not prove.

28. Phase 17: turn a kernel into a reusable backend

A fast kernel is like a fast kitchen appliance. It is not enough for the blender blades to spin quickly if someone must unpack, assemble, and wash the whole machine for every smoothie. Before Phase 17, ChoiceForge repeatedly did small pieces of setup around an already-fast generated kernel.

A compiled plan is a checked instruction packet

Phase 17 creates a **compiled plan**. The plan remembers:

- the exact equation recipe, identified by a cryptographic hash;
- which inputs are whole columns and which are single numbers;
- the data type and storage slot for every input;
- how skim lookups are indexed;
- the compiled CUDA kernel; and
- the coefficient matrix already stored on the GPU.

Reusing a plan is safe only when the next batch has the same shape of meaning. The row values may change. A price or coefficient value may change. But a column cannot quietly become a scalar, a floating-point value cannot quietly become an integer, and two different columns cannot quietly acquire a new shared-memory layout. If any of those rules changes, ChoiceForge refuses that plan and builds or selects a matching one. This is called **fail closed**: when the program is uncertain, it does not guess.

There is a subtle scalar rule too. Imagine two constants both happen to equal 2 today. Combining them into one slot would be wrong if tomorrow one becomes 3 and the other becomes 7. Persistent mode therefore gives semantic scalar inputs stable slots even when their current values are equal.

Why trip mode choice had to join the GPU path

An early experiment made destination faster but made the later trip-mode step about 2.4 seconds slower. The code had not created work from nowhere. It had stopped Sharrow from warming up during destination, so Sharrow paid its first compilation cost later during mode choice. This is a **displaced cost**: one timer looks better because the cost moved to another timer.

Phase 17 adds a continuation bridge. The same generated FP32 CUDA utility plans now serve trip mode choice. ActivitySim still owns the nested-logit probabilities, random numbers, final choices, and result tables. ChoiceForge replaces only the utility calculation. If the generated path encounters an unsupported case, it records the reason and calls the original Sharrow path. The reported runs had zero fallbacks.

The small exactness gate

On 1,001 public households, the final Phase 17 qualification checked:

Check	Result
Real destination batches	30 of 30 exact
Rows evaluated	85,126
Feature values	32,262,754 of 32,262,754 exact
Utility values	1,787,646 of 1,787,646 exact
Reused destination plans	20 calls
Reused trip-mode plans	10 calls

Check	Result
GPU fallbacks	0
Changed modeled decisions	0

Two saved columns are diagnostics rather than later decision inputs. Their last decimal places can differ because the declared FP32 GPU policy does not promise identical decimal text to Sharrow:

Diagnostic	Largest difference	Allowed gate
Destination logsum	0.000008	0.0001
Mode-choice logsum	0.00000191	0.00001

The output checker gives tolerance only to those two named columns. Every trip destination, trip mode, and other modeled output still has to match exactly.

The five-pair 50,000-household result

The clean proof used five fresh baseline/candidate pairs and fixed both BLAS thread settings at 16. A prior 24-thread attempt suffered a native OpenBLAS failure in a baseline process, so that incomplete series was discarded.

Measure	Baseline median	Phase 17 median	Result
Trip destination	28.4 s	27.3 s	1.040x faster
Complete model	191.474 s	190.307 s	1.006x faster

Every one of the five destination runs was faster than every baseline destination run. The five paired destination savings were 1.8, 0.8, 0.9, 0.8, and 1.9 seconds. A bootstrap calculation put the middle saving's 95% interval between 0.8 and 1.9 seconds. That is strong component-level evidence.

The complete model is more complicated. Its paired savings were 1.845, -0.042, 1.360, -0.038, and 2.595 seconds. Three won and two lost by less than one twentieth of a second. The median improved by 1.167 seconds, but the 95% bootstrap interval ran from -0.042 to 2.595 seconds. Because that interval still touches a small slowdown, the strict whole-model superiority gate does not pass. “The median was faster” is true; “whole-model superiority is proven” would be too strong.

Reusable buffers: useful, but still experimental

Creating GPU arrays also costs time. Phase 17 implements an optional workspace that lets a checked plan reuse its device input and output storage. A first version used pinned host memory. It uploaded faster but packed columns more slowly, so it was rejected. The retained version keeps fast NumPy column packing and reuses only device allocations.

That version passed the exact gate. In one 50,000-household diagnostic it reduced measured destination packing plus upload time from about 1,471.5 to 1,364.1 milliseconds, a saving of roughly 107 milliseconds. However, one pair cannot prove a whole-model effect, and unrelated initialization varied by more than two seconds in that run. Reusable buffers therefore remain an opt-in experiment, not part of the five-pair headline proof.

29. What the result means and what comes next

Here is the honest Phase 17 conclusion:

- the generated FP32 GPU backend exactly matches its published FP32 CPU answer key on every qualified feature and utility cell;
- all reported modeled decisions match the reference;

- repeated trip-destination superiority is proven on this RTX A4000 and public MTC workload;
- the five-run whole-model median is faster, but the strict whole-model proof is not yet complete; and
- the result has not yet been replicated on a second GPU or a different public ActivitySim model.

The fastest path to a stronger claim is to collect more independent, controlled 50,000-household pairs with the same locked software, hashes, and 16-thread setting. That narrows the uncertainty around the tiny whole-model losses. The most important scientific step after that is external replication: run the same exactness and A/B protocol on a second NVIDIA GPU and then on a second public ActivitySim model. A result that survives different hardware and different travel equations is much more useful to the ActivitySim and Sharrow communities than a single impressive number from one workstation.

30. Phase 18: can a chain of model steps live on the GPU?

After Phase 17, the obvious ambitious question was: what if the GPU did not calculate one isolated piece and then hand everything back to the CPU? What if household data entered the GPU once, several dependent model steps happened there, and only the finished answers came back?

Phase 18 builds the first honest version of that idea. It is not a whole travel model yet. It is a **vertical slice**, meaning a narrow but complete path from real input rows through multiple calculations to final outputs. Imagine building one fully working elevator before constructing every floor of a skyscraper. The elevator proves the important machinery can work together, but it does not mean the building is finished.

Does “GPU-only” really mean the CPU does absolutely nothing?

No. A regular computer must use its CPU to start Python, read a CSV file, read configuration, tell the GPU which kernel to launch, handle errors, and write a result file. Even a CUDA program is launched by host code.

In this project, **GPU-native modeled execution** has a precise meaning:

- The CPU may read input and upload one partition before modeling starts.
- The CPU may launch kernels and wait for them.
- The CPU may download final outputs after modeling finishes.
- The CPU may not calculate a utility, random choice, logsum, or modeled total.
- The program may not secretly use NumPy or pandas when a GPU operation is missing.

The runtime closes a gate called `seal_ingress`. After that gate closes, a host array entering a modeled stage, an intermediate modeled result leaving the GPU, or a CPU fallback causes a hard error. This behavior is called **fail closed**. If the program cannot honor its promise, it stops instead of quietly producing a misleading benchmark.

What is GPU state?

A travel model changes tables over time. A household first gets an auto ownership result. A person later gets a work location. Tours and trips are then created. Later steps depend on earlier answers. All those current tables are the model's **state**.

Phase 18 adds a `DeviceTable`. Its columns must be CUDA arrays, which means the actual numbers live in GPU memory. All columns must describe the same number of rows. A `GpuNativeRuntime` owns these tables and records what happens at the CPU/GPU boundary. It counts input bytes, output bytes, modeled transfers, CPU fallbacks, and kernel stages.

Why random numbers are part of correctness

Travel models use random draws to turn probabilities into simulated choices. Suppose household 42 gets random number 0.63 in one run. If changing the batch size gives it 0.18, the model can change even though no travel assumption changed. That would make scaling unsafe.

The new GPU random generator uses three stable labels:

```
entity ID + project seed + stream ID -> the same random draw
```

The entity ID identifies the household or person. The project seed identifies the run. The stream ID identifies the decision, such as first choice or second choice. A hash thoroughly mixes those integers, and the GPU turns part of the result into a number between zero and one.

The important fact is what the formula does **not** use: it does not use the row's position inside a partition. Household 42 therefore gets the same bits whether it is processed in one table of 2.8 million households or a small table of 250,000. A separate NumPy implementation checks the GPU generator bit for bit.

Why adding numbers can be nondeterministic

Floating-point addition is not perfectly associative. In exact mathematics, `(a + b) + c` equals `a + (b + c)`. On a computer, rounding after each addition can make the last bits different.

Many fast GPU totals use **atomics**: many threads race safely to add their values to one total. The final answer can depend on which thread arrives first. Phase 18 instead sorts group IDs and gives one thread responsibility for each group. That thread adds rows in a fixed order. It may not be the fastest possible future method, but its behavior is easy to explain and reproduce.

What the public-data vertical slice does

The benchmark reads all 2,875,192 households from the public Prototype MTC table. It then performs this chain after the GPU boundary closes:

- **Stage 1:** Build eight household features, such as normalized household size, workers, automobile count, income, and zone information.
- **Stage 2:** Generate the first stable random stream on the GPU.
- **Stage 3:** Make a fused choice among 21 alternatives.
- **Stage 4:** Put that first answer into the inputs of a second choice.
- **Stage 5:** Generate a different stable stream and make the dependent second choice.
- **Stage 6:** Sort households by traffic analysis zone, or TAZ, and total the first choices within each zone.
- **Stage 7:** Download the final answers for checking.

The word **dependent** matters. The second choice really uses the first choice. This is not a collection of unrelated kernels that happen to run one after another.

Which parts are real, and which are invented for the test?

The households and their fields come from the real public MTC benchmark. The 21 alternatives and eight-feature calculation have the shape of travel-model choice work. However, the coefficients are fixed synthetic test values. Nobody estimated them from a survey, so these choices must never be interpreted as a forecast of real behavior.

The data transform also publishes its assumptions:

- Negative income codes mean missing for this systems test and become zero.
- Income above $250,000 is capped before normalization.

- Very large household, worker, and automobile counts are capped before normalization so rare coding outliers do not dominate the test.
- Household ID is the stable random key.
- The calculation uses a declared 32-bit floating-point policy.

These choices make a stable systems benchmark. They do not claim to be the right behavioral specification for a transportation agency.

31. Phase 18 results: fast, reproducible, and carefully bounded

The full-table benchmark used the local NVIDIA RTX A4000, driver 571.59, Python 3.11.14, nine measured repetitions, and a fused parallel Numba CPU comparison. Compilation warm-up happened before the measured repetitions.

Full public household table	Median time
Parallel fused Numba CPU	0.457023 seconds
GPU modeled work only	0.031357 seconds
GPU including one upload and final downloads	0.055327 seconds

That is **14.575 times faster** for modeled computation and **8.260 times faster** even after the allowed boundary transfers.

This is a much larger speedup than the whole ActivitySim results reported in earlier phases because the scopes are different. Phase 18 measures a compact GPU-native vertical slice where almost every operation is parallel. It does not include CSV parsing, all ActivitySim components, checkpoint work, or output writing. A person comparing the numbers must keep that difference in mind.

What exactly reproduced?

The same GPU calculation ran in two arrangements:

- all 2,875,192 households together; and
- consecutive partitions containing at most 250,000 households each.

Every final GPU choice was identical. Every final GPU logsum had identical bits. This is the Phase 18 **replication guarantee**: scaling the workload into deterministic partitions does not change the GPU result.

Runtime telemetry also showed:

Boundary check	Result
Modeled CPU fallbacks	0
Host-to-device modeled bytes after ingress closed	0
Device-to-host modeled bytes before final output	0
Sampled active GPU allocation peak	about 452.4 MiB

The memory number samples active CuPy arrays after stages. A very short-lived temporary allocation could be higher, so it is a measured lower bound rather than a perfect hardware high-water mark.

Why are CPU and GPU answers not all bit-identical here?

The Numba CPU and CUDA GPU both implement the same mathematical model, but they do not promise the same lowest-level versions of exponential and fused-multiply-add operations. Across all 2,875,192 rows:

CPU/GPU comparison	Observed difference
First choices	1 row
Dependent second choices	3 rows
Largest dependent logsum difference	0.007143
Largest zone-total difference	1.0

One first-stage boundary choice can affect the second-stage feature, which is why the number grows from one to three. The observed rates are 0.348 and 1.043 differences per million rows. They pass the published limits of one and two per million.

This is not the same guarantee as Phases 14-17, where a specially defined CPU recipe and generated CUDA utility recipe matched feature and utility bits exactly. Phase 18 deliberately compares against a strong normal Numba choice implementation. Its cross-architecture claim is **numerical equivalence within written bounds**, while its GPU partition claim is **bit-exact**.

During development, a first full-table run used `log1p(income)`. NumPy and CUDA rounded that function differently by one float32 unit on some rows, and one row landed close enough to a choice boundary to change. The project did not hide the failure or simply rerun until it disappeared. It replaced the feature with capped linear normalization, documented the arithmetic policy, reran the full table, and still reported the remaining CUDA-versus-Numba boundary cases.

What fits in this GPU's memory?

The local RTX A4000 reports 16,376 MiB of GPU memory. The public MTC skim file contains 826 datasets. If every dataset is counted as its raw uncompressed array, the collection needs 13.389 GiB. Earlier real 50,000-household ActivitySim integration runs already peaked near 8.4 GiB.

Therefore the complete future model should not load every skim and every state table at once. Phase 18 proposes four planned pools:

Planned use	Budget
CUDA, driver, and failure reserve	2 GiB
Frequently used, or “hot,” skims	4 GiB
Persistent model state	2 GiB
Largest component workspace	3 GiB
Remaining safety and partition room	about 4.99 GiB

A **hot skim cache** keeps the travel-time maps needed soonest on the GPU and evicts others safely when space is needed. A **population partition** processes a stable set of households and everything belonging to them, then moves to the next set. These are design budgets, not yet measured full-model limits. The maximum safe production partition cannot be claimed until the missing model components have real memory high-water measurements.

What does Phase 18 prove?

- A sealed GPU state boundary can be enforced rather than merely promised.
- Stable GPU random draws survive changes in partition size.
- Dependent choices and an ordered group total can stay on the device.

- This vertical slice can process the entire public household table at once.
- The GPU is substantially faster than fused parallel Numba for this work, including the boundary transfers.
- Every declared performance, numerical, partition, and fallback gate passes.

What does it not prove?

- It is not a complete ActivitySim replacement.
- Its synthetic coefficients do not make calibrated travel predictions.
- It does not prove every person, tour, trip, timetable, shadow-price, and skim table fits together in 16 GB.
- It does not yet provide restartable GPU checkpoints.
- It does not accelerate file parsing or report writing.
- It does not prove the same speed on another GPU or model.
- It does not claim that ordinary CPU and GPU math is bit-identical.

32. The practical path from this slice to a whole GPU model

The next-generation project is possible on this GPU if it grows in dependency order and treats memory and replication as design rules.

Phase A: finish the GPU table toolbox

Implement indexed joins, filters, stable sorting, scatter and gather, category encoding, group operations, and missing-value policies. Each operation needs a small readable CPU oracle, adversarial tests, and an explicit arithmetic rule.

Phase B: build the hot skim cache

Track which skim arrays each component needs. Upload them asynchronously, retain frequently reused arrays, and evict only after every dependent CUDA event finishes. Record cache hits, misses, transferred bytes, and memory peaks.

Phase C: port one calibrated household-person chain

Replace synthetic coefficients with a real public specification. Preserve stable entity channels and compare every intermediate column, not only the final output. A missing GPU operation must stop qualification rather than use a hidden CPU fallback.

Phase D: make restart behavior safe

Long models must recover after a failure. Either serialize device tables at declared checkpoints or make a documented host checkpoint boundary. Restoring a checkpoint must reproduce random streams and every downstream table.

Phase E: add tours, trips, timetables, destinations, and shadow prices

These are harder because they create variable numbers of rows, use large skim lookups, and update shared state. Port them one dependency layer at a time. Measure high-water GPU memory after every component and reduce partition size before an out-of-memory failure becomes possible.

Phase F: run the real whole-model proof

Only after the calibrated chain is complete should the project run fresh, interleaved CPU/GPU whole-model trials. The proof must include table hashes, random-stream checks, fallback counters, memory peaks, transfer bytes, component times, whole-model time, software hashes, and a second-machine replication.

The implication is exciting but specific. This workstation has enough GPU to build and test a serious next-generation model and to run large partitions very quickly. It does not have enough memory to hold the public model's entire skim collection plus every future state object without caching and partitioning. Success therefore comes from a GPU-native **system** - state, randomness, memory, checkpoints, and kernels together - not from writing one spectacular kernel.

33. Phase 19: replace the invented equations with a real calibrated chain

Phase 18 answered an engineering question: can several dependent calculations stay on the GPU and run quickly? Its answer was yes, but its choice equations used invented test coefficients. Phase 19 asks the harder scientific question:

> Can the GPU run real published travel-model equations and reproduce the > choices that ActivitySim already made?

The answer for the first calibrated household-to-person chain is **yes**.

What does “calibrated” mean?

A travel-model equation combines facts such as income, household size, age, location, and travel time. Each fact gets a number called a **coefficient**. A coefficient says how strongly that fact moves a choice up or down.

For example, this made-up equation is only an illustration:

```
score for owning two cars
  = 0.4 × number of workers
  + 0.2 × income in thousands
  - 0.3 × transit accessibility
```

Real modelers estimate coefficients from observed travel and household data. When a model uses those estimated numbers, it is **calibrated**. Phase 19 uses the real coefficients published with Prototype MTC Extended. It does not invent new behavior.

Which two real decisions run on the GPU?

The first model is **auto ownership**. Each household chooses one of five answers: zero, one, two, three, or four-or-more automobiles.

The second model is **mandatory tour frequency**. A *tour* is a journey that starts at home, visits one or more places, and returns home. A mandatory tour goes to work or school. A person can choose among these five answers:

- one work tour;
- two work tours;
- one school tour;
- two school tours; or
- both a work and a school tour.

Only people whose earlier daily-activity choice says they have a mandatory activity enter the second model. In the public checkpoint, that is 78,900 of 132,536 people.

The chain is truly connected:

```
50,000 households
        |
        v
GPU auto-ownership model
        |
        | join each answer to people in that household
        v
78,900 mandatory-person choosers
        |
        v
GPU mandatory-tour-frequency model
```

The second model uses the **new GPU auto answer**. It does not secretly copy ActivitySim's saved auto answer into the calculation. The saved answer is used only afterward as an answer key.

What is a checkpoint replay?

A long video game lets you save at a checkpoint instead of starting from the beginning after every test. ActivitySim can also save its tables after model steps. Phase 19 starts from public saved tables immediately before the chosen components.

This is powerful because the input state is fixed. The CPU reference, the GPU, and ActivitySim's saved output all begin from the same households, people, zones, work locations, school locations, accessibility values, and earlier daily-activity answers.

It also limits the claim. Phase 19 does **not** recalculate school location, work location, or the earlier daily-activity model on the GPU. Those are frozen inputs. This is a two-component calibrated replay, not yet a whole-model run.

34. How Phase 19 makes a real choice

Both components use a method called **multinomial logit**, or MNL. The name is less important than its four steps.

Step 1: evaluate expressions

An **expression** turns input columns into a useful model feature. Examples in the real auto-ownership file include:

- whether the household has exactly two drivers;
- household income, capped within a range;
- retail accessibility by car or transit;
- whether the home is in San Francisco County; and
- estimated automobile time savings per worker.

The auto model has 29 active expressions. The mandatory-tour model has 98. Together, the GPU evaluates 127 published expressions.

The safe expression reader understands only operations that were reviewed and tested, such as addition, comparison, Boolean “and/or,” clipping a value to a range, and choosing one of two values with `where`. If an unfamiliar operation appears, the run stops. It never sends the expression to unrestricted Python code and never quietly falls back to the CPU.

Step 2: calculate utilities

For every alternative, each expression value is multiplied by its calibrated coefficient. The products are added to form a **utility**. Utility is a model score, not money or electrical service. A higher utility means the alternative is more attractive to the model.

With 50,000 households and five auto alternatives, the first component makes 250,000 utility values. With 78,900 people and five tour-frequency alternatives, the second makes 394,500 utility values.

Step 3: turn utilities into probabilities

MNL converts the five utilities in a row into five probabilities that add to one. A numerically safe version first subtracts the largest utility, then uses the exponential function, and finally divides each result by the row total.

Suppose the probabilities were:

```
alternative:  A     B     C     D     E
probability: 0.10  0.25  0.40  0.20  0.05
```

These probabilities describe ranges on a line from zero to one. A random draw of 0.52 lands in C's range, so C is selected.

A **logsum** is the logarithm of the total exponentiated utility. Modelers use it as a summary of how attractive the complete choice set is. Phase 19 keeps logsums on the GPU and downloads them only as final diagnostic outputs.

Step 4: use ActivitySim's exact random draw

Matching probabilities is not enough. Two programs can have the same probabilities but choose different answers if they use different random draws.

ActivitySim creates a stable seed from four items:

```
run seed + channel name + model-step name + household/person ID
```

It then uses an algorithm called **MT19937**, from NumPy's older `RandomState`, to create the draw. Phase 19 implements the needed part of MT19937 directly in a CUDA kernel. The GPU produces the exact same 64-bit floating-point draw as NumPy for every tested household and person.

Phase 19's kernel supported the first draw in a model step, called offset zero. These two public components need exactly that. Phase 20, explained below, has since implemented and proved later offsets as well.

How does a person find the right household answer?

Tables do not always have rows in matching order. A person's row contains a `household_id`. The GPU must find the household row with that ID.

Phase 19 adds a GPU **keyed join**. It sorts source IDs, searches for every requested ID, checks that no ID is missing, and gathers the matching values. The same tool also looks up land-use and accessibility facts by zone ID.

This is a basic database operation performed on the GPU. It is essential for a whole travel model because household, person, tour, trip, and zone tables constantly refer to one another by IDs.

35. How we proved the result and what it means

The proof uses three separate answer layers.

1. A plain NumPy version independently calculates expressions, utilities, probabilities, ActivitySim random draws, and choices.
2. That CPU version must exactly reproduce ActivitySim's saved checkpoint choices.
3. The GPU is compared with every important CPU intermediate and separately with ActivitySim's saved final choices.

This avoids circular reasoning. The GPU is not declared correct merely because another function that shares all its code agrees with it.

Correctness results

Public-checkpoint test	Result
Households	50,000
All people in input state	132,536
Mandatory-person choosers	78,900
CPU auto choices different from ActivitySim	**0**
GPU auto choices different from ActivitySim	**0**
CPU tour-frequency choices different from ActivitySim	**0**
GPU tour-frequency choices different from ActivitySim	**0**
Expression feature differences	**0**
Random-draw bit differences	**0**
Largest utility difference	about `0.0000000000000018`
Largest probability difference	about `0.00000000000000044`
Choice differences across nine repeated GPU runs	**0**

Those tiny utility and probability differences come from low-level floating-point library arithmetic. They are many orders of magnitude below the written limits and did not change a single choice.

Speed results

The program first warmed up compilation. It then measured nine complete CPU and GPU replays on the RTX A4000.

Measured boundary	Median time	Relative speed
Independent CPU replay	0.458724 seconds	1.000×
GPU modeled work	0.025713 seconds	**17.840× faster**
GPU with input upload and final download	0.037997 seconds	**12.073× faster**

The transfer-inclusive number is the most practical result for this checkpoint replay. It includes moving the input tables to the GPU once and bringing the final choices and logsums back. It does not include reading Parquet files from disk or running all the upstream and downstream ActivitySim components.

Did the GPU secretly use the CPU?

No modeled fallback was recorded. After the GPU boundary closed:

Forbidden event	Recorded amount
CPU modeled fallbacks	0
Late host-to-GPU modeled transfers	0 bytes
Early GPU-to-host modeled transfers	0 bytes

The CPU still launches kernels and checks a few true/false error flags. That is control work, not a second model calculation.

Does Phase 19 make the first 18 phases useless?

No. It makes Phase 19 the best **headline**, but it could not stand alone as a trustworthy project.

Earlier phases built and tested the choice kernels, strict expression language, float rules, ActivitySim adapters, public checkpoints, failure policies, device-state runtime, and transfer counters. They also studied destination and scheduling components that Phase 19 does not run. Phase 18 proved that a dependent GPU state chain could work before real calibrated behavior was added.

The right summary is:

- Phase 19 replaces Phase 18's synthetic behavioral demonstration for this household-to-person boundary.
- Phases 1-18 remain the engineering foundation, audit trail, and evidence for other component types.
- Phase 19 still does not replace a full fresh ActivitySim CPU/GPU comparison.

36. Phase 20: make a real tour table on the GPU

Phase 19 ended with one frequency answer per mandatory person. Phase 20 asks a different kind of computer question: how do we create a table when different input rows create different numbers of output rows?

One person may choose one work tour. Another may choose two school tours. A third may choose one work tour and one school tour. This is called **variable-length table expansion**.

Imagine students lining up for a school photo. Some need one chair and some need two. Before assigning chairs, the organizer counts how many each student needs. A **prefix sum** adds the counts from left to right:

```
chairs needed:        1  2  1  2
starting chair:       0  1  3  4
total chairs needed:             6
```

The GPU uses the same idea. It counts each person's tours, calculates the starting output position, allocates exactly 81,983 rows, and runs one fused kernel that writes all columns.

What is in a tour row?

A tour row needs much more than “work” or “school.” It contains:

- the tour's own stable ID;
- the owner person's and household's IDs;
- whether it is a work or school tour;
- which tour of that type it is;
- which mandatory tour should be scheduled first;
- how many mandatory tours that person has;
- the home origin and work or school destination;
- its category; and
- its participant count.

That is 12 columns. Phase 20 compared every generated value, not just a sample or the row count, with ActivitySim's public saved table. Every column had zero mismatches across all 81,983 rows.

A subtle work-and-school rule

ActivitySim physically stores work rows before school rows. But for a person who is not classified as a worker and has both tours, school must be scheduled first. The row order stays the same while the schedule number is swapped.

This distinction sounds tiny, but getting it wrong changes the person's timetable and can affect later choices. ChoiceForge has a focused test for the rule and also checks it across the complete public table.

37. Why IDs must be boring and predictable

A random-looking but stable tour ID lets later model steps find the same tour, attach a repeatable random stream, and compare two runs.

The public model has 41 possible tour labels when mandatory, optional, joint, and at-work tours are considered together. The mandatory labels occupy fixed positions:

Tour label	Position among 41 labels	ID rule
First school tour	31	person ID × 41 + 31
Second school tour	32	person ID × 41 + 32
First work tour	39	person ID × 41 + 39
Second work tour	40	person ID × 41 + 40

For example, if person 100 has a first work tour, its ID is `100 × 41 + 39 = 4,139`.

ChoiceForge checked the formula against every public mandatory tour. More importantly, the generated ID set exactly equaled the set of tour IDs consumed by the next scheduling component. That proves the two stages connect, instead of merely producing two separately correct-looking results.

38. What mandatory-tour scheduling decides

After making a work or school tour, the model must decide when it starts and ends. A **tour departure-and-duration alternative**, shortened to **TDD**, is one possible time window. Examples might be “leave at 7, return at 17” or “leave at 9, return at 15.” The public model has 190 base TDD alternatives.

Not every window is possible for every tour. A later tour cannot overlap a time already occupied by an earlier tour. Its scores can depend on:

- the person's worker or student type;
- income and household size;
- travel time to work or school;
- whether the destination is downtown;
- the start, end, and duration of the proposed window;
- how attractive the available travel modes are at those times;
- the previous tour's end time; and
- open blocks in the person's timetable.

This is why scheduling is a harder downstream test than another five-answer frequency model.

The full public capture

Phase 20 resumed the preserved 50,000-household ActivitySim pipeline exactly after mandatory-tour frequency, ran only mandatory scheduling, and captured:

- 81,983 real tour choosers;
- six first/second work, school, and university groups;
- 15,242,743 feasible chooser-time rows;
- published coefficients and expressions;
- real time-dependent mode-choice logsums;
- real timetable facts;
- ActivitySim's random draws; and
- ActivitySim's selected time positions.

The compact format does not save a huge table with every expression already calculated. It saves shared ingredients once and lets the generated CPU and GPU programs evaluate the expressions. The six compressed files total about 12 megabytes.

39. The gate failed first, and that was success

The first full run did **not** pass. Three GPU scheduling choices and one CPU reference choice differed among 81,983 tours.

This was not dismissed as “only a few.” Every mismatch occurred where the random draw was extraordinarily close to the boundary between two cumulative probabilities. The investigation found that the older kernel changed the arithmetic recipe:

- ActivitySim keeps its random draw as a 64-bit floating-point number.
- The old kernel rounded that draw to 32 bits.

- ActivitySim normalizes its 32-bit probability weights before subtracting them from the draw.
- The old kernel compared the rounded draw with unnormalized weights.

The corrected kernel keeps the draw at 64 bits, creates normalized 32-bit probabilities, and subtracts them in the same alternative order as ActivitySim. The independent CPU answer key was corrected to use NumPy's exact 32-bit probability-reduction behavior too.

A new regression test places a draw just one 64-bit step above `0.5`. Rounding it to 32 bits changes the answer, so the old bug cannot quietly return.

After the correction:

Proof check	Result
CPU schedule choices different from ActivitySim	**0 of 81,983**
GPU schedule choices different from ActivitySim	**0 of 81,983**
CPU choices different from GPU choices	**0 of 81,983**
GPU TDD values different from public checkpoint	**0 of 81,983**
Largest CPU/GPU logsum difference	about `0.00000381`
Choice differences across nine GPU repeats	**0**

The small logsum difference is reported honestly. The CPU and GPU add many 32-bit weights using different reduction trees, so their last few bits need not match. The behavioral choice output is exact.

40. Phase 20 speed results

Compilation was warmed before nine measurements on the RTX A4000.

Work measured	CPU median	GPU median	Speedup
Create tour table, data already on GPU	0.006904 s	0.000601 s	**11.496×**
Create tours including upload and ID download	0.006904 s	0.001101 s	**6.272×**
Schedule kernel, compact data already on GPU	0.173748 s	0.009601 s	**18.097×**
Schedule kernel including compact transfers	0.173748 s	0.059196 s	**2.935×**

These are meaningful wins at their named boundaries. They do not mean that the complete travel model is now 17.8 times faster.

The scheduling timer starts after ActivitySim has prepared the mode-choice logsums, feasible time alternatives, and timetable facts. Those preparation steps remain on the CPU. In simple language, the GPU now cooks the scheduling recipe very quickly, but ActivitySim still gathers and measures several major ingredients.

41. Random streams can now continue and restart

A repeatable simulation sometimes needs the first random number for an entity, then the second or the 313th. Phase 19 implemented only the first number and correctly refused anything else.

Phase 20 implements any nonnegative **random offset** for ActivitySim's MT19937 generator. Tests check offsets 311, 312, and 313 because that crosses an internal state-refresh boundary where a careless implementation often fails. Every tested GPU double matches NumPy bit for bit. The common first-draw case still uses its faster specialized kernel.

A **random ledger** records how many draws each table and model step has used. Saving that ledger prevents a restarted model from accidentally reusing the first random number and changing its scientific result.

Phase 20 also writes an audit manifest. For each tour column it records the row count, data type, and SHA-256 fingerprint. It records completed components, the selected TDD fingerprint, the random offset, and the capture fingerprint. This makes silent changes detectable.

It is not yet a fully self-contained restart file because the compact scheduling-preparation arrays live beside it rather than inside it.

42. What did Phase 20 prove, and what remains?

Phase 20 proves that a calibrated GPU choice can create a correctly shaped dependent table, give its rows ActivitySim-compatible identities, and feed all of them into a real downstream calibrated choice kernel. This is more than a standalone arithmetic demonstration.

It still does not prove a complete GPU-only ActivitySim model. The frozen upstream location and CDAP inputs from Phase 19 remain. Scheduling preparation still uses the CPU. Other tour and trip components remain outside this chain. Only one NVIDIA GPU architecture and one public model have been used for this new proof.

The next major phase should move scheduling preparation itself:

1. Keep the needed travel-time and travel-cost skim arrays in a managed GPU cache.
2. Calculate time-dependent mode-choice logsums on the GPU.
3. Represent each person's timetable as device state.
4. Construct the feasible TDD alternatives from that timetable.
5. Schedule first tours, update the timetable, then schedule later tours.
6. Feed those arrays directly into the already qualified scheduling kernel.
7. Compare the entire scheduling component, including preparation, with a fresh ActivitySim run.
8. Repeat on another NVIDIA architecture and a second public model.

The long-term lesson is that raw GPU arithmetic was never the only problem. The difficult and valuable work is preserving identities, table growth, random streams, time state, maps, arithmetic rules, restart evidence, and exact behavior while data crosses a chain of decisions. Phase 20 closes the table- growth and downstream-kernel parts of that problem. Scheduling preparation is now the clearest next frontier. Phase 21, explained next, completes that frontier at an honest compact-cache boundary and proves the raw-skim CUDA side separately.

43. Phase 21: prepare real schedules on the GPU

Phase 20 began with a prepared list of possible times. That was useful, but it left a fair question: what if preparing the list takes much longer than choosing from it?

Phase 21 moves that preparation work. For each person, the GPU stores a small 21-slot timetable. Think of it as a row of boxes, one box for each modeled hour. A box can be empty, mark the start of a tour, mark its end, or mark time in the middle of a tour.

For every tour, the GPU considers 190 possible combinations of leaving and returning. It asks whether each combination collides with something already on the person's timetable. It keeps the feasible combinations and throws away the colliding ones. This creates a list of different length for every tour.

The compact computer format for those different-length lists is called **CSR**, short for compressed sparse row. You do not need to memorize the name. It is simply:

```
one long list of all feasible time choices
+ a small list saying where each person's part begins and ends
```

CSR matters because the GPU does not need to reserve 190 output rows for every tour after it knows many choices are impossible.

Why tours must be scheduled in order

A person may have two mandatory tours. The second tour cannot be checked until the first tour has occupied part of the timetable. Phase 21 therefore uses a careful mixture of parallel and sequential work:

- thousands of people and alternatives are processed together inside a batch;
- the first-tour batch finishes and updates the timetable; and
- only then does a later-tour batch begin.

This is not a weakness. It is the causal rule of the model. Running later tours too early would be fast but scientifically wrong.

44. How 190 time choices become a 25-number cache

Network travel times do not change separately for every modeled hour. The public MTC model groups hours into five broad skim periods, such as early, morning, midday, afternoon, and evening.

A tour has an outbound period and an inbound period. Five possibilities in each direction make a 5-by-5 table, or 25 slots. Because a tour cannot return before it leaves, only 15 slots are used for a first tour.

This lets Phase 21 replace millions of repeated mode-logsum values with one small cache per tour. A **logsum** is a single number summarizing how attractive all the travel modes are for that outbound and inbound period pair. It is not a simple average: attractive modes contribute more, and related modes are grouped by a nested-logit formula.

The cache builder does not assume repeated values are equal. It checks their 32-bit patterns. If two hourly alternatives mapped to the same slot contain different values, the build stops with an error.

The original captured prepared arrays used 518,909,174 bytes in memory. The Phase 21 primitive arrays use 12,688,620 bytes, **40.896 times smaller**. The six compressed files on disk total about 6.3 megabytes.

45. What exactly the Phase 21 speed test includes

The stopwatch begins with the compact 5-by-5 cache and per-tour facts. It includes all six real scheduling batches and all of these operations:

1. checking the timetable for collisions;
2. counting feasible alternatives;
3. building the CSR list;

4 deriving the previous tour's end time;

5 calculating all seven timetable-dependent ActivitySim facts;

6 gathering the right mode logsum;

7 calculating calibrated scheduling utilities and probabilities;

8 using ActivitySim's original 64-bit random draw;

9 selecting the TDD; and

10 updating the timetable before the next batch.

Here, **TDD** means a tour departure-and-duration alternative. It identifies a start time and end time from the public 190-row alternative table.

Nine measurements on the RTX A4000 produced:

Qualified work	Compiled CPU median	GPU median	GPU advantage
Data already resident	0.214878 s	0.021069 s	**10.199x**
Primitive transfer included	0.214878 s	0.024755 s	**8.680x**

Including primitive transfers added about 3.7 milliseconds to the GPU median and reduced its advantage from 10.199x to 8.680x. This is why the report keeps resident and transfer-inclusive boundaries separate instead of hiding data movement. Both measured GPU boundaries were much faster than the same compiled CPU reference.

The benchmark also lists ActivitySim's 23.338-second scheduling-component time for context. It does **not** divide that number by 0.021 seconds to claim a thousand-fold improvement. ActivitySim's timer includes raw mode-logsum work, pandas tables, and workflow management that are outside this stopwatch.

46. How Phase 21 proves the answer is the same

Speed is accepted only after the answer passes independent checks.

The public workload contains:

Item	Count
Households	50,000
Persons with timetable rows	78,900
Mandatory tours	81,983
Sequential scheduling batches	6
Possible TDD alternatives	190 per tour
Feasible rows actually regenerated	15,242,743

A readable CPU version and a compiled parallel CPU version serve as answer keys. The GPU must match the captured ActivitySim CSR offsets, feasible TDD IDs, eight row values, and adjusted chooser values. It then must match every selected TDD. A second GPU run must produce the same result again.

The final counts are simple:

- CPU preparation mismatches: **0**;
- GPU preparation mismatches: **0**;

- CPU TDD mismatches: **0 of 81,983**;
- GPU TDD mismatches: **0 of 81,983**;
- CPU-versus-GPU TDD mismatches: **0**; and
- differences on the repeated GPU run: **0**.

The evidence file also stores the machine and software versions, all nine raw timing samples, source fingerprints, input fingerprints, per-batch counts, and a restart/checkpoint fingerprint. A fingerprint is a SHA-256 hash: a long label that changes if the bytes change.

The GPU-only method rejects a NumPy or pandas modeled array. This prevents a hidden CPU fallback from accidentally being timed and described as GPU work.

47. The live raw-skim gate and the one-choice mystery

The compact-cache benchmark is the strongest speed result, but where does the cache come from? Phase 21 also answers that question with a real ActivitySim integration run.

The live path reads the public network skim tensors. A **tensor** is a multi-dimensional array. Here its coordinates can include origin zone, destination zone, and time period. The generated GPU program evaluates the public tour-mode specification for 21 travel alternatives and then performs the nested-logit reduction.

Six live calls processed 1,210,124 rows. Every call used CUDA, none fell back to the CPU, and the utility matrix stayed on the GPU when it entered the nest reducer.

The gate did not pass on the first try.

First, the compiler did not understand expressions asking for a reverse trip, the maximum of forward and reverse distance, and special round-trip skim directions. It stopped safely instead of guessing. Phase 21 added those exact operations and tests that prove a reverse lookup swaps origin and destination without uploading a second copy of the network.

Next, all GPU calls ran, but one of 81,983 schedules changed. A fresh CPU-only control matched the frozen reference, so the GPU arithmetic needed more work.

The cause was a tiny but real recipe difference:

- Sharrow's utility calculation uses 32-bit numbers and permits a fused multiply-add operation.
- The strict ChoiceForge proof compiler used separate multiplication and addition and deliberately forbade fusion.
- ActivitySim then promotes the utilities to 64-bit numbers for a particular sequence of nested exponent, sum, and logarithm operations.

A **fused multiply-add**, often written FMA, calculates a x b + c as one hardware instruction with one final rounding step. Separate multiplication and addition round twice. Usually the difference is far too small to matter. This one tour had a random draw extremely close to a probability boundary, so it did matter.

Phase 21 did not weaken the old strict contract. It added a separate, explicitly named Sharrow-compatible fused policy for this live path. It also added a CUDA nest reducer that follows ActivitySim's real mixed-precision tree. An attempted all-32-bit nest policy changed 1,203 schedules and was rejected.

After the correct policies were combined, the live result was:

Live proof check	Result
CUDA calls	6
CUDA mode-logsum rows	1,210,124

Live proof check	Result
CPU fallbacks	0
Utility download/re-upload before nesting	0 bytes
TDD differences	**0 of 81,983**
Start-time differences	**0**
End-time differences	**0**

This failed-first history is part of the evidence. It shows why "almost every choice" was never treated as enough.

48. What Phase 21 means, and what still needs to be done

Phase 21 proves two major facts:

- from a compact time-period logsum cache, the GPU can regenerate the complete calibrated mandatory-scheduling preparation and choice boundary exactly and about ten times faster than compiled parallel CPU code; and
- from real public raw skims, the generated CUDA utility and nest engine can feed ActivitySim without changing any mandatory schedule.

These are complementary proofs, not yet one continuous production timer. The live ActivitySim API still creates pandas tables and materializes the compact cache between the raw-skim GPU work and the standalone device-resident scheduler. Joining those two already qualified halves is the next major step.

After that, a larger program of work remains:

1. checkpoint the live GPU timetable so a stopped run can restart exactly;
2. port non-mandatory, joint, and at-work scheduling;
3. build a managed hot cache for the 13.389-GiB public skim collection;
4. partition the population without changing identities or random draws;
5. repeat the proof on another NVIDIA GPU;
6. repeat it on a second public activity-based model; and
7. run repeated complete-model CPU/GPU pairs before claiming whole-model superiority.

Phase 21 does not make Phases 1 through 20 unnecessary. Those phases created the public captures, independent CPU answers, random-number rules, stable tour IDs, expression compiler, device handoff, precision tests, and fail-closed habits that made Phase 21 provable. A large success that cannot be reproduced would be a demonstration. This project is trying to build evidence.

49. Phase 22: join the two fast halves

Phase 21 ended with two strong but separate results. One result started with raw road and transit data and calculated mode-choice logsums on the GPU. The other started with a small logsum cache and prepared and chose schedules on the GPU. Phase 22 connects them.

The connected path now works like this:

1. ActivitySim supplies a batch of tours, their stable IDs, and network skim lookups.
2. Generated CUDA code calculates 21 mode utilities for each representative time combination.

3. A CUDA nested-logit reducer turns those utilities into one logsum per combination.
4. The logsum stays on the graphics card and is placed into a small cache for its tour.
5. The GPU timetable rejects schedule choices that collide with an earlier tour.
6. The GPU evaluates the scheduling rules, uses ActivitySim's exact random draw, chooses a departure-and-duration alternative, and marks those hours busy before the next batch begins.
7. Only final schedule labels return to ActivitySim for the normal path.

There are six ordered batches because first tours must be scheduled before second tours. Phase 22 completed all six for 81,983 mandatory tours.

50. Why one answer was still incredibly hard

The first connected GPU run was wrong for exactly one tour. The reference chose TDD 169, while the GPU chose 168. This was not ignored as a harmless rounding difference.

That tour's random number was only about **0.00000000085** beyond a probability boundary. Imagine drawing a line with a thick marker and asking whether a dust speck is on the left or right edge. Several mathematically reasonable computer recipes can round the line by a few billionths and put the speck on different sides.

The investigation found several hidden parts of ActivitySim's recipe:

- Sharrow builds 65 32-bit features, including two temporary features whose coefficients are zero.
- It performs a 32-bit dot product. A **dot product** multiplies matching pairs and adds all the products.
- Because of one ActivitySim safety setting, it exponentiates the utilities without first subtracting the largest utility.
- NumPy's CPU exponential and sum do not promise the same final bits as CUDA's GPU exponential and parallel sum.

All of those methods are numerically sensible. But “sensible” is not the same as “guaranteed to make the identical simulated choice.” A simulation is a chain: one changed schedule can change a person's later availability and then change more decisions.

Hard-coding 169 would have been cheating. It would only memorize the test. The solution instead asks the GPU how close every random draw is to the nearest choice boundary. If the distance clears the guard tested on this public model, the GPU result is accepted. If it is very small, the row is marked ambiguous and the exact ActivitySim/Sharrow recipe decides it. A different model or GPU must test the guard again; it is not a universal law of floating-point math.

51. Is Phase 22 completely GPU-only?

No - and that distinction matters.

Most of the expensive work is on the GPU, and the bulk modeled logsums are not downloaded. But 57 of 81,983 tours were close enough to a probability boundary to use the exact CPU/Sharrow adjudicator. That is **0.0695%** of the tours. The adjudicator downloaded 11,400 bytes, about eleven kilobytes, per full run.

This is a hybrid with a tiny, measured correctness exception. Calling it “absolutely no CPU” would be inaccurate because ActivitySim still organizes the workflow, owns the random numbers, writes output tables, and resolves those 57 ambiguous choices.

Why keep this exception? Because removing it before CUDA and Sharrow share an identical definition for dot products, exponentials, sums, and rounding would trade a true exact result for a marketing slogan. A future Sharrow GPU backend or a dedicated expression compiler can remove the resolver, but only after it passes this boundary test and every other saved proof gate.

52. The final Phase 22 evidence

The team ran three fresh CPU/GPU pairs from the same public 50,000-household checkpoint. Each run included raw network skim setup, ActivitySim orchestration, all six scheduling batches, and output writing for the resumed component.

Pair	CPU time	GPU-connected time	CPU / GPU
1	42.358 seconds	36.599 seconds	1.157x
2	40.389 seconds	31.963 seconds	1.264x
3	40.250 seconds	32.030 seconds	1.257x

The GPU path was faster in every pair. The middle paired speedup was **1.257x**. Comparing the middle CPU and GPU times gives **1.261x**. These are smaller than the 8.680x-to-10.199x compact-kernel results because the live timer includes shared setup and ActivitySim work that still runs on the CPU. Both measurements are useful; they answer different questions.

Every live GPU run produced the same proof facts:

- 6 of 6 batches joined;
- 1,210,124 raw-skim rows used generated CUDA;
- 0 CUDA fallbacks;
- 57 exact boundary checks;
- 11,400 boundary-transfer bytes;
- 0 changed random draws;
- 0 changed TDD choices out of 81,983;
- 0 changed start times;
- 0 changed end times; and
- a restart record containing fingerprints of the selected TDDs and final timetable.

The result means this machine can make the complete resumed mandatory-tour scheduling component faster, starting from raw public skims and ending with exact schedules. It does not yet mean the whole ActivitySim model is faster or GPU-only. Non-mandatory, joint, at-work, trip, and destination components still need the same connected proof. A second GPU and a second public model are also needed before claiming that the result is portable.

Most importantly, Phase 22 shows why the earlier phases were not wasted. The one-in-81,983 boundary problem was found because the project preserved random draws, stable identities, CPU references, public checkpoints, arithmetic policies, and fail-closed tests. Those are the pieces that turn “the GPU looked fast” into a result another reviewer can challenge and reproduce.

53. Why the 1.257-times result was not the GPU's real limit

The Phase 22 timer measured a whole ActivitySim component. That was the right test for compatibility, but most of its roughly 32 seconds were not spent in the scheduling kernel. ActivitySim had to restart a saved pipeline, create and join pandas tables, organize model calls, synchronize state, validate results, and write output. Making a small kernel even faster could not remove that surrounding work.

This is an example of **Amdahl's law**. It says that a system cannot become much faster by improving a part that occupies only a small fraction of total time. If a one-second calculation sits inside 30 seconds of unchanged setup, making that calculation instantaneous still leaves about 30 seconds.

The escape is to change who owns the workflow. Phase 23 uploads the needed state once and lets several dependent model stages share it on the GPU.

54. What a device-resident runtime is

Think of a school group project stored in a shared online folder. If every student downloads the entire folder, changes one sentence, emails it back, and waits for someone else to upload it, most time is spent moving and organizing files. A better system keeps the current document in one shared place and records which person changed which section.

Phase 23 does the same for model data. A **device table** is a set of columns in GPU memory. The runtime gives every table a name and version. Every model stage must declare which tables it reads and writes. It follows these rules:

- input tables may be uploaded before the runtime is sealed;
- after sealing, modeled outputs must be GPU arrays;
- a host NumPy array or hidden CPU fallback causes a hard failure;
- all outputs of a stage are checked before any become official;
- replacing an existing table must be requested explicitly;
- temporary tables are released after their final user;
- only named final columns may be published; and
- checkpoints record enough actual data to restart, not only a success label.

This still needs a CPU process to read files and launch CUDA kernels. "Device resident" means the modeled state and arithmetic stay on the GPU between the declared entry, publication, and checkpoint boundaries.

55. The six connected modeled stages

The public Phase 23 graph begins with 50,000 households and 132,536 persons. It performs six connected kinds of work:

1. The calibrated auto-ownership model chooses the number of cars for every household.
2. The mandatory-frequency model uses those new car choices and decides which mandatory people make one or two work or school tours.
3. The GPU creates the variable number of tour rows.
4. Stable tour IDs connect every generated tour to its scheduling row.
5. Six ordered scheduling batches test time choices, calculate probabilities, and select TDDs.
6. The timetable is updated so later tours cannot overlap earlier tours.

The graph ends with 78,900 mandatory-person choices and 81,983 scheduled tours. The compact 5-by-5 mode-logsum caches are uploaded inputs in this phase. Phase 22 proved that CUDA can generate them from raw skims, but joining that producer inside the sealed Phase 23 graph remains future work.

56. Two optimizations that only a resident runtime can use well

The first optimization is a **compiled join map**. Households repeatedly look up land-use data by zone, and persons repeatedly look up household data. The older code sorted and searched those keys every run. Phase 23 validates the relationships once and keeps the resulting row numbers on the GPU. Later runs use direct array indexing.

The second optimization is a **fused fixed-choice compiler**. The mandatory frequency model has 98 published expressions and five possible answers. The old GPU path created 98 feature columns, multiplied them by coefficients,

then ran separate probability and choice operations. The new generated CUDA kernel does all of this for one person in one thread:

- read the person's permanent fields;
- insert the auto-ownership answer made by the previous GPU stage;
- evaluate 98 expressions;
- accumulate five utility scores;
- calculate probabilities and a logsum; and
- use the exact ActivitySim random draw to select the answer.

It changed zero of 78,900 frequency choices. Its largest logsum difference from the independent dense CPU calculation was less than `0.0000000000000001`.

57. The Phase 23 proof

The team started three independent Python and CUDA processes. Each process ran nine measured CPU repetitions and nine measured resident-GPU repetitions after both compilers were warmed.

Result	Process 1	Process 2	Process 3	Middle result
CPU modeled time	0.7673 s	0.7605 s	0.7677 s	0.7673 s
GPU resident time	0.0313 s	0.0353 s	0.0315 s	0.0315 s
Resident speedup	24.516x	21.555x	24.405x	**24.405x**
Setup-inclusive speedup	1.356x	1.314x	1.360x	**1.356x**

"Setup-inclusive" charges the one-time input upload, scheduler creation, device join-map compilation, one modeled run, and final publication to only one run. It excludes file reading and compiler warm-up on both CPU and GPU. Optional checkpoint writing is reported separately and took about 0.34 seconds.

If the same graph is run repeatedly, the fixed setup cost is shared. Arithmetic using the measured medians gives 9.055x over ten repeated runs and 20.868x over one hundred. These are amortized calculations, not measurements of different policy scenarios; measured parameter-batch scenarios are still future work.

Every proof gate passed in all three processes:

- 0 changed auto-ownership choices;
- 0 changed mandatory-frequency choices;
- 0 differences in all 12 generated tour columns;
- 0 changed tour IDs or TDDs;
- 0 changed timetable cells;
- 0 repeat differences across 27 measured GPU runs;
- 0 post-seal modeled transfers;
- 0 CPU fallbacks; and
- 0 schedule differences after checkpoint restore.

58. What this success means - and what it does not mean

Phase 23 proves that the GPU's large advantage was hidden by the old CPU-owned workflow. When several calibrated components share resident state, the modeled chain is more than 20 times faster in every independent process on this machine. Even a single run remains faster after paying the measured setup and publication costs.

It does not prove that the entire ActivitySim model now runs in 0.03 seconds. Earlier workplace/school location and CDAP state are frozen inputs. Scheduling mode-logsum caches are also inputs. Destination choice, non-mandatory tours, joint tours, at-work subtours, trips, shadow pricing, and normal pipeline output still need device-resident implementations.

Phase 24 completed skim-cache management, and Phase 25 has now bound the real utility/logsum producer to that cache. Remaining work is to create the dense chooser rows inside the versioned runtime, connect the generated caches to its timetable stage, represent sampling and shadow-pricing state as versioned tables, then port the other model components. The proof must eventually be repeated on another GPU and another public model.

The important change is that the project now has both pieces: a compatible ActivitySim path that proves exact integration, and a resident runtime that proves the architecture can deliver large speedups. Compatibility explains today's result; residency shows the path to the next generation.

59. Phase 24: keeping the useful road-network facts on the GPU

A travel model repeatedly asks questions such as, "How long is the drive from zone 42 to zone 900 in the morning?" The answer lives in a **skim**. You can picture a skim as a giant spreadsheet: origins are rows, destinations are columns, and each cell contains a time, distance, toll, fare, or other network measurement. A time-dependent skim is a stack of five spreadsheets for early morning, morning peak, midday, afternoon peak, and evening.

The public MTC file stores 826 matrices. As uncompressed 64-bit numbers, they would occupy about 13.39 GiB and nearly fill this 16 GiB card before model state and workspace are added. Loading everything would be fragile.

Phase 24 builds a **hot cache**: the frequently used data. It reads the reviewed expression recipe, called the strict IR, and discovers required skim names automatically, avoiding errors in a hand-typed list.

The 315-term recipe contains 209 logical bindings. "Logical" means a meaning in an equation, such as outbound time or the reverse-direction inbound time. One reverse binding can use the same physical cube with its origin and destination swapped. Shared directional views reduce this to 149 physical allocations.

60. Why 6.38 GB fits when 13.39 GiB did not

The original matrices use **float64**, which stores each number in 8 bytes. The qualified GPU equations use **float32**, which stores each number in 4 bytes. Float32 has less precision, so a model may use it only when the numeric contract and choice tests allow it; it is not a free change.

The required source values total 12,757,865,000 bytes. Their float32 GPU form is exactly half: 6,378,932,500 bytes, or about 5.94 GiB. Phase 24 declares an 8 GiB budget first. Too much data, a missing matrix, or a wrong shape causes an error instead of silent data loss.

The `.omx` file is only about 0.734 GB because HDF5 compression saves disk space, not expanded working memory. Comparing GPU allocation only with the compressed file size would be misleading.

The cache uploads each physical cube once. The runtime then attaches those existing GPU arrays without making a second copy, gives them a named table and version, and seals entry. No new modeled host array can enter afterward.

61. How every raw read was checked

The proof uses all six real mandatory-scheduling logsum batches. They contain 1,210,124 tour-and-time rows. A small number refer to destination zone 0, ActivitySim's missing-destination marker. Those cannot be legal matrix positions, so the proof reports and excludes 5,530 of them. That leaves 1,204,594 valid public OD/period rows.

For each valid row, both sides read all 209 bindings: 251,760,146 values per run. Instead of downloading that giant table, each feeds every exact 32-bit pattern through two 64-bit **hashes**, or compact fingerprints. A changed input is overwhelmingly likely to change them; source-file and matrix hashes add protection against accidental corruption.

Three new Python/CUDA processes each ran five measured repetitions:

Result	Process 1	Process 2	Process 3	Middle result
CPU time	5.691 s	5.662 s	5.669 s	5.669 s
GPU resident time	0.029 s	0.069 s	0.028 s	0.029 s
Resident speedup	193.114x	82.323x	205.639x	**193.114x**
Upload-inclusive speedup	1.813x	1.851x	1.802x	**1.813x**

GPU timing varied with memory clocks and system state, so all values are reported. Even the slowest was 82.323 times faster. All output fingerprints matched; 15 GPU repetitions were exact; post-seal modeled transfers and CPU fallbacks were zero.

The huge 193.114-times number is for this isolated memory-access and hashing job. It must not replace Phase 23's 24.405-times calibrated chain result or Phase 22's 1.257-times live ActivitySim component result. Different timing boundaries answer different questions.

62. What Phase 24 changes, and the honest next step

Before Phase 24, a bounded hot-skim cache was only a plan. It is now measured on public data, protected by a hard budget, and registered inside the sealed runtime. The raw values are bit-exact, fast to access, and small enough to leave space for later model state.

It does **not** yet calculate the real 315 mode-choice terms from those cubes. It does not perform the 21-mode nested-logit calculation, produce the 5-by-5 scheduling logsum cache, or replace that precomputed input in Phase 23. The all-binding hash program is a proof instrument, not a travel-choice model.

This was the seven-job plan for the next connected phase:

1. Create the real chooser and OD/period row indices on the GPU.
2. Give these resident cube pointers to the already-qualified strict CUDA expression plan.
3. Evaluate 315 terms for 21 travel modes.
4. Reduce those utilities through ActivitySim's nested-logit tree.
5. Scatter the resulting logsums into each tour's 5-by-5 scheduling cache.
6. Define reproducible arithmetic for the 57 choices near probability boundaries, or retain and measure an explicit adjudication boundary.
7. Replace Phase 23's saved logsum input and rerun every proof gate.

Phase 25 completed jobs 2 through 5 and removed the bulk saved-logsum input from its new producer. Job 1 remains because ActivitySim still prepares dense rows and coordinates. Job 6 remains as the explicit 57-row adjudication

boundary. Job 7 is partly complete: the replacement producer is proven, but the historical Phase 23 timetable benchmark has not yet been rewired to consume it directly. The next section explains the measured result.

63. Phase 25: connecting the road facts to the real equations

Phase 24 was like placing all the needed road maps on a workbench. It proved that the GPU could find and read the right cells quickly, but it did not yet use those cells to predict a travel mode.

Phase 25 adds the missing calculator. For every tour-and-time possibility, it does the following:

1. Read facts about the traveler and the tour.
2. Read time, distance, toll, fare, and transit facts from the resident skims.
3. Evaluate 315 real expressions from the public MTC model.
4. Turn those expression values into scores for 21 travel modes.
5. Combine related modes using the model's nested-logit tree.
6. Produce one logsum that summarizes how attractive the available modes are.
7. Place that logsum in the correct cell of a 5-by-5 time-period cache used by tour scheduling.

The six programs represent work, school, and university tours for the first and second mandatory tour. Together they process 1,210,124 rows. That means 381,189,060 expression evaluations and 252,915,916 logical skim reads in one complete replay.

64. What "sealed and resident" means here

The first time a batch appears, ActivitySim has already prepared its traveler fields and origin, destination, and time-period numbers. Phase 25 freezes those inputs in GPU memory. It also keeps the compiled equation program, coefficients, output space, and raw skim arrays there.

There was one subtle trap. To place each new logsum in a scheduling cache, the computer needs a map saying, for example, "source row 12,345 belongs to tour 700 and morning-to-evening cache cell 9." An early passing version rebuilt that map on the CPU during every replay. The values stayed on the GPU, but the operation was not truly resident.

The final version compiles this map once. About 19.36 MB of device positions remain on the GPU, along with reusable output caches. During a measured replay, the CPU does not rebuild the map, upload it, or receive the modeled logsums. It only launches the GPU work.

The device state for this boundary includes:

Resident item	Approximate size
149 unique skim arrays	6.20 GB
Dense equation inputs	348.5 MB
Origin/destination/time coordinates	154.9 MB
Compiled cache-placement map	19.4 MB

The six programs share the 149 skim arrays. They are not six separate 6.20 GB copies. Counting GPU pointers across the whole process proves that only 149 physical arrays exist.

65. How the Phase 25 result was proved

One fast run can be luck. The proof starts three fresh Python and CUDA processes. Each process first runs the real ActivitySim mandatory-scheduling component, captures the six actual programs, and checks final tour times against the frozen public answer. Then it warms the resident path once and measures five complete replays.

Result	Process 1	Process 2	Process 3	Middle result
Resident time	0.169039 s	0.169739 s	0.169423 s	**0.169423 s**
Speedup versus initial live CUDA setup/execution	10.389x	9.655x	9.475x	**9.655x**
Changed logsum bits across five replays	0	0	0	**0**
Changed final TDD/start/end values	0	0	0	**0**

Fifteen measured resident replays therefore produce 18,151,860 logsums in total, with zero changed bits. All three live ActivitySim runs also reproduce all 81,983 mandatory tour schedules exactly.

The 9.655-times number needs a careful label. It compares the sealed replay with the **initial CUDA path**, which has to resolve input bindings, pack and upload dense fields, prepare plans, and then run the same GPU mathematics. It does not compare with a CPU calculation. It also does not include the rest of ActivitySim. Earlier phase numbers answer those different questions.

66. What success means, what remains, and why it matters

The resident producer no longer needs a saved 5-by-5 logsum cache as its bulk input. It creates those cache values from real raw network skims and real calibrated equations. This closes the largest missing mathematical gap between the Phase 23 resident model graph and the Phase 24 raw-data layer.

However, the older Phase 23 benchmark remains a historical artifact with its original saved-cache input. Phase 25 proves the replacement producer; the next phase must wire its generated caches directly into the versioned timetable stage and regenerate dense chooser rows on-device.

The complete live path is also not absolutely CPU-free. ActivitySim still prepares the six dense batches. Python launches kernels and writes outputs. Most importantly, 57 scheduling draws are extremely close to a probability boundary. Tiny differences in 32-bit and 64-bit arithmetic could change which side of the boundary wins. The GPU detects these rows without looking at the saved answer, downloads 11,400 bytes of raw logsums, and lets the exact ActivitySim/Sharrow calculation settle them. Only one row is known to change without that protection, but all 57 are treated conservatively.

The next success should therefore have two parts:

- move dense row construction and generated logsum caches directly into the versioned resident scheduling graph; and
- define one shared Sharrow/CUDA rule for utility arithmetic, exponentials, ordered sums, and probability search so all 57 boundary rows can stay on the GPU with exact final choices.

After that, the project can extend the same pattern to non-mandatory tours, joint tours, destinations, trips, and shadow pricing. The practical lesson is that large GPU gains do not come only from making multiplication faster. They come from keeping a connected chain of real work and its data in one place, while proving that every published decision still means the same thing.

67. Phase 26: the calculator now feeds the calendar directly

Phase 25 ended after it filled each tour's small 5-by-5 box of mode-choice summary numbers. Phase 26 connects that box to the next job instead of saving it, sending it through the CPU, or loading an old copy.

The connected GPU assembly line now does this:

1. Read the already loaded road and transit facts.
2. Evaluate the six real 315-part mode-choice equations.
3. Combine 21 travel modes into one summary number for each possible time.
4. Place each number in the correct tour-and-time cache cell.
5. Compare each tour with 190 possible start/end schedules.
6. Reject schedules that overlap something already on that person's calendar.
7. Build the surviving row numbers and their index on the GPU.
8. Choose a schedule using the model's fixed random number.
9. Mark that schedule as occupied in the person's GPU calendar.
10. Repeat in the required order for all six batches, then publish final TDDs.

TDD means **tour departure and duration**. It is simply the model's numbered label for one start-time/end-time pair.

The large table in step 7 is not loaded from a saved file. The GPU creates the collision mask, counts the surviving alternatives, computes where each person's rows begin, and writes the row fields itself. Across the six batches, this corresponds to 15,242,743 feasible scheduling rows.

68. Why 57 choices were unusually difficult

Imagine a random draw landing almost exactly on a line between two slices of a pie chart. A microscopic rounding difference can put the point on the left slice on one computer and the right slice on another.

That happened for 57 of 81,983 tour choices. The GPU could identify them, but the older live system downloaded 11,400 bytes of their numbers and asked the original CPU/Sharrow calculation to decide. Only one of the 57 actually chose a different schedule without that check, but treating all 57 cautiously made the guarantee stronger.

The investigation found the exact source. Sharrow first builds 65 32-bit expression values. It then asks Numba to multiply a 65-item row by a two-dimensional coefficient column. Repeating that exact array shape matches all 190 captured Sharrow utility values bit-for-bit. A mathematically equivalent one-dimensional dot product does not necessarily add the products in the same order.

Next, ActivitySim applies 32-bit exponentials, adds them using NumPy's order, divides by the total, and walks through the probabilities. The captured probability vector was reproduced bit-for-bit on the CPU. CUDA's exponential and addition machinery is allowed to round differently, however. Knowing the CPU recipe does not automatically make a different GPU math library emit the same last bit.

69. The honest Phase 26 solution: a qualified GPU decision map

Phase 26 removes the **runtime CPU trip** without claiming that CUDA and NumPy have identical arithmetic.

Before the benchmark is sealed, the 57 delicate public cases and Sharrow's correct TDD answers are qualified and versioned. The answer map is then kept in GPU memory. During a replay, the GPU independently detects which rows are close to a boundary and uses the resident answer for those rows. All 57 stay on the GPU; one answer is changed; zero boundary bytes go to the CPU.

This is like a teacher-approved correction card for 57 known trick questions. It gives an exact, fast, repeatable answer for this fixed public benchmark. It is not permission to change the questions and keep the same card. If the model equations, coefficients, population, road data, random draws, or row order change, the card must be checked and rebuilt.

A more general future solution would define one arithmetic rule shared by Sharrow and CUDA: exactly how expressions round, exactly how exponential is computed, exactly how values are added, and exactly how the random draw is compared. That would let new scenarios prove themselves without a list of known delicate cases.

70. What was measured and what it proves

Three fresh Python and CUDA processes ran the public 50,000-household model. Each warmed once and then ran the complete resident assembly line five times.

Result	Process 1	Process 2	Process 3	Middle result
Complete resident time	0.199694 s	0.205299 s	0.200852 s	**0.200852 s**
Measured replays	5	5	5	**15**
Mode-logsum rows each replay	1,210,124	1,210,124	1,210,124	exact
Changed logsum bits	0	0	0	**0**
Final tour-time mistakes	0	0	0	**0**
Delicate rows kept on GPU	57	57	57	**all**
Boundary bytes downloaded	0	0	0	**0**

The test covers 81,983 mandatory tours. Only the final 163,966 bytes of TDD labels are published after the modeled graph finishes. The runtime recorded no post-seal modeled upload, no intermediate modeled download, and no modeled CPU fallback.

The 0.201-second number starts after roughly 6.8 GB of GPU state is already loaded and the six dense mode-choice input batches already exist. It does not include loading road matrices, asking ActivitySim to prepare those dense inputs, compiling kernels, or writing output files. It must not be compared directly with a full cold CPU run.

For fair GPU-versus-CPU evidence, keep the older matched comparisons:

- the GPU scheduling preparation/choice kernel was 10.199 times faster than its compiled CPU implementation at the same boundary;
- the paired live raw-skim mandatory-scheduling component was 1.257 times faster end to end; and
- the calibrated resident vertical slice was 24.516 times faster than its modeled CPU baseline.

Phase 26 proves something different and important: once the data and programs are resident, the real raw-skim-to-calendar chain can stay connected, finish in about one fifth of a second, reproduce every published mandatory-tour time, and avoid the last tiny runtime CPU adjudication.

What remains is to generate the six dense mode-choice chooser fields and origin/destination/time coordinates from higher-level resident household, person, tour, land-use, and timetable tables. Python still launches the graph, and the project still covers only the components already listed in this guide. Non-mandatory tours, joint tours, destinations, trips, shadow pricing, and ordinary ActivitySim output writing remain future work.

71. Phase 27: stop carrying the same row facts again and again

Phase 26 was extremely fast, but it began with about half a gigabyte of already prepared row arrays. Many rows repeated the same facts. A tour's home zone, destination, income, and vehicle facts do not change just because the model is testing another possible departure time. Time-related facts also repeat across many tours.

Phase 27 asks a simple question: can the GPU rebuild the large arrays from the small facts instead of storing every repeated copy?

It uses four kinds of compact fact:

1. A **constant** is one value shared by the entire batch.
2. A **tour value** is one value shared by all tested times for one tour.
3. A **time-slot value** is one value shared by the same exact start/end pair.
4. A **response pattern** is a reusable short list. For example, a parking cost can depend on both a tour's parking rate and the duration of the tested schedule. Tours that produce the same short list share one pattern number.

The rows are ragged: different tours can have different numbers of feasible alternatives. A compact row-offset list says where each tour begins and ends. The GPU uses that list to recover both the row's owner and its position inside the tour without storing a large value column.

72. How we prevent compression from changing an answer

Calling something "compressed" is not enough. If the compression rounds a number or guesses a pattern, it could change a probability and eventually a travel choice.

The Phase 27 compiler checks the raw computer bits of every floating-point input, integer input, origin, destination, and time coordinate. A column is accepted only if it exactly fits one of the four forms above. A response pattern is accepted only if its dictionary is smaller than the original column. There is no secret option to keep an unexplained full row column.

After building the compact form, CUDA reconstructs every array and compares all of its bits with ActivitySim's original. If one bit differs, sealing fails. The proof also records the addresses of the original arrays and fails if any of those addresses appears in the timed runtime.

This is why a field called `daily_parking_cost` caused useful early failures. It was not simply one value per tour or one value per time. The public model calculates it from both. The response-pattern representation handles that real relationship without pretending it is simpler than it is.

73. The Phase 27 speed and memory result

Three fresh processes each measured five complete graph replays. They also used five warm-ups and five measurements for the reconstruction-only CPU/GPU comparison.

Result	Process 1	Process 2	Process 3	Middle result
Complete compact-input-to-calendar graph	0.205337 s	0.208764 s	0.204956 s	**0.205337 s**
GPU reconstruction	0.002915 s	0.002906 s	0.002939 s	**0.002915 s**
NumPy reconstruction	0.490051 s	0.491203 s	0.499566 s	**0.491203 s**
CPU time divided by GPU time	168.13x	169.04x	169.96x	**168.52x**

The old captured row arrays occupied 503,411,584 bytes. The compact persistent facts occupy 25,042,522 bytes, a **20.102-times reduction**. The GPU still needs about 508.3 MB of reusable workspace because the existing 315-term equation kernels expect the reconstructed row layout. Workspace is overwritten each run; it is not another saved input copy.

The complete Phase 27 graph is only 2.23% slower than Phase 26. That is the important architectural result: rebuilding half a gigabyte of exact inputs adds only about 0.0045 seconds to the middle complete-graph measurement.

Every one of the 15 full replays processed 1,210,124 mode-logsum rows. No logsum bit changed. No final TDD changed. No captured row pointer was retained. No modeled data moved to or from the CPU after sealing, and no modeled CPU fallback ran.

74. What 168.52x means - and what it does not mean

The 168.52-times result compares one clearly matched job: materialize the same 503.4 MB of arrays from the same compact factors using NumPy or CUDA. It does not compare a full CPU ActivitySim model with a full GPU model.

The 0.205337-second number is also a warm resident boundary. Raw skim cubes, compact facts, compiled programs, and timetable state are already on the GPU. It excludes cold file loading, initial factor discovery, compilation, and ordinary output writing.

The older speed results still answer other questions. In particular, Phase 22 measured a 1.257-times live ActivitySim component improvement, and Phase 23's calibrated resident vertical slice measured a 24.516-times modeled CPU/GPU advantage. Phase 27 should not replace their labels with its much larger but narrower reconstruction number.

75. What comes next

Today ActivitySim still prepares the large arrays once so Phase 27 can learn and prove the compact representation. The timed graph no longer needs those arrays, but a cold production startup still does.

The next compiler should create compact facts directly from resident household, person, tour, land-use, and scheduling-alternative tables. Named expressions should replace anonymous response dictionaries where their arithmetic can be made identical. ActivitySim's dense arrays would remain only as a qualification answer key, not a production prerequisite.

Two other frontiers remain. The fixed public benchmark still uses the 57-entry GPU decision map for arithmetic-boundary cases; a shared Sharrow/CUDA exponential, summation, normalization, and search rule would generalize that guarantee to changed scenarios. The connected GPU coverage must also expand from mandatory tours to non-mandatory tours, joint tours, destinations, trips, and the rest of a complete ActivitySim workflow.

76. Phase 28: replace remembered patterns with named rules

Phase 27's response dictionaries were like compact answer sheets. They were exact, but they said, "tour pattern 7 produces this list" rather than saying why the list had those values. That is safe for a fixed qualified run, but a new road condition should cause a new answer because of a rule, not because a new answer sheet happened to be prepared.

Phase 28 replaces every one of those special columns with a named rule:

- parking cost equals a tour's hourly rate times the tested duration;
- toll-mode availability checks whether the matching toll skims are positive;
- walk-transit availability checks the matching outbound and return transit skim values; and
- drive-transit availability makes the same checks and also requires a car.

There are 15 rules: one parking-cost rule and 14 availability rules. The compiler carries each source's name from the model recipe into the GPU plan. If it meets an unexplained response column, it stops. It does not quietly keep the old dictionary.

77. Why parking cost needed a small math detective story

ActivitySim multiplies the parking rate and duration with a high-precision number, then stores a 32-bit result. Suppose you only see a rounded answer such as 1.23. Dividing 1.23 by one duration may not recover the original rate well enough to reproduce a different duration's last bit.

The compiler therefore finds an **interval** of possible high-precision rates for every observed rounded cost. It intersects all those intervals and chooses a rate that regenerates every observed 32-bit result exactly. If no such rate exists, compilation fails.

This is stronger than guessing, but it is still a qualification technique. A future production loader should read the original unrounded parking rate from the land-use table and combine it with the tour's free-parking fact directly.

78. How we tested that the rules are not just memorizing one run

The public 50,000-household benchmark is essential, but repeating the same input cannot by itself show that a formula responds correctly to change.

We created five extra test worlds. Each has different people, car ownership, parking rates, possible times, zones, and raw road/transit numbers. Some skim values are positive, some zero, and some negative so availability rules must change. A separate readable NumPy calculation makes the answer key.

Across 8,000 rows, the CUDA generator matches every bit. All 15 rules are actually exercised, no response dictionary remains, and the five output hashes are different. This proves the input generator reacts to changed ingredients. It does **not** claim that five complete new ActivitySim policy scenarios have been qualified from beginning to end.

79. The Phase 28 result and tradeoff

Result	Process 1	Process 2	Process 3	Middle result
Complete semantic-input-to-calendar graph	0.211799 s	0.210766 s	0.216390 s	**0.211799 s**
Named input generation	0.008788 s	0.008805 s	0.008802 s	**0.008802 s**
Checkpoint-to-result ActivitySim run	31.853 s	32.148 s	31.170 s	**31.853 s**

The original captured arrays occupy 503,411,584 bytes. Phase 28 keeps 20,258,882 bytes of compact input state, a **24.849-times reduction**. That is 19.102% less compact state than Phase 27, and 5,439,864 bytes of response dictionaries are gone.

The semantic graph is not faster than the dictionary graph. Its middle full time is 3.147% above Phase 27 and 5.450% above Phase 26. That is expected: a dictionary lookup is cheaper than gathering many real transit skims and re-evaluating availability. The gain is stronger generality, smaller state, and a clear path to new scenarios, not a new whole-model speedup claim.

All 15 public replays still process 1,210,124 mode-logsum rows and publish all 81,983 mandatory-tour time labels exactly. No modeled data crosses the CPU/GPU boundary after sealing, and no modeled CPU fallback runs.

80. What Phase 28 still does not finish

The timed graph no longer needs dense input rows or response dictionaries. However, ActivitySim still creates dense rows before sealing so the compiler can discover and check constant, per-tour, and time-slot facts. The parking rate is recovered from qualified outputs rather than loaded from raw land use.

The next phase should build those compact facts directly from resident household, person, tour, land-use, and alternative tables. ActivitySim's dense rows should become a test-only answer key. That would remove the cold-start dense-row dependency, not merely the timed dependency.

After that, the other major correctness frontier remains: replace the frozen 57-case boundary map with a shared Sharrow/CUDA definition for exponential, addition order, normalization, and probability search. Coverage can then grow to non-mandatory and joint tours, destinations, trips, and complete model output.

81. Phase 29: stop learning the recipe from the answer sheet

Imagine that a student solves a problem correctly, but only after studying a completed answer sheet and noticing repeated patterns. Phase 27 did something similar when it compressed prepared rows. Phase 28 replaced the most obvious remembered patterns with named rules, but other values were still classified by looking at the prepared result.

Phase 29 writes down where **every** ingredient comes from before checking the answer:

- one raw row for each tour supplies facts about the person and household;
- the land-use table supplies facts about origins and destinations;
- the model constants supply items such as walking speed and maximum wait;
- controlled random inputs supply the stochastic ride-hail ingredients;
- possible start/end times supply alternative slots; and
- resident skim cubes supply road and transit conditions.

For each real program, this makes 57 declared sources. If the compiler meets a column, period, skim direction, or zone that it cannot explain, it stops. It does not inspect the dense answer and invent a new category.

82. What "one row per tour" changes

One tour may be tested against roughly 25 representative time choices. The old preprocessor therefore repeats the same age, car ownership, household size, destination, and similar facts many times. With more than a million rows, that repetition becomes expensive.

The Phase 29 source table has one row per tour. A small GPU expansion step uses the tour owner and the tested time slot to create exactly the row needed by the utility kernel. Origin, destination, outbound time, return time, and reversed skim directions are also generated from named coordinate rules.

The important proof is not merely that the result is smaller. The compiler reads **zero bytes** from the dense oracle while constructing the plan. Only after construction does the test harness compare the generated arrays with ActivitySim's old arrays, byte for byte.

83. Parking and availability now come from upstream causes

Phase 28 had to recover a parking rate from rounded parking-cost answers. Phase 29 reads the original hourly rate from the destination's land-use row. If a work tour has free workplace parking, its rate is zero. CUDA multiplies that unrounded rate by the tested duration.

Phase 29 also expands from 14 to all 18 availability rules. Four road-mode fields happened to be constant in the public benchmark, but that does not mean they will remain constant after a network change. They now check raw SOV, HOV2, HOV3, and tolled skim values just like the other named rules.

This is a useful scientific lesson: a value being constant in one dataset is not the same as a law saying it must always be constant.

84. How we tried to break the raw-table compiler

The public benchmark was run in three fresh processes, with five complete resident replays in each process. That makes 15 full replays. Every one of 1,210,124 mode-logsum rows keeps the same final bits, and all 81,983 scheduled tour times remain exact.

We also created ten changed test worlds:

- five raw-table populations contain 10,000 tours with changed people, households, zones, parking prices, free parking, density, value of time, and nonzero ride-hail wait variation; and
- five raw-skim worlds contain 8,000 rows with changing positive, zero, and negative road/transit values.

A separate readable implementation calculates each answer. All direct source formulas and all 18 availability formulas pass, and every scenario hash is different. These are focused source/formula tests, not ten complete regional policy simulations.

85. The Phase 29 result, cost, and honest next boundary

Result	Process 1	Process 2	Process 3	Middle result
Complete raw-table-input-to-calendar graph	0.210148 s	0.225311 s	0.226075 s	**0.225311 s**
Raw input generation	0.010439 s	0.010328 s	0.010506 s	**0.010439 s**
Checkpoint-to-result ActivitySim run	30.647 s	30.759 s	31.021 s	**30.759 s**

Phase 29 keeps 24,849,394 bytes instead of 503,411,584 bytes of repeated rows, a **20.259-times reduction**. It uses 22.659% more compact state than Phase 28 because values that happened to be constant are now stored per tour so a changed population can change them. The complete graph is 6.380% slower than Phase 28. That is the price measured on this machine for a much stronger, scenario-capable source contract.

The qualification process still lets ActivitySim create dense rows. They are now only an independent answer key and a way to expose the existing compiled utility interface; they are not inputs to the Phase 29 compiler and do not enter the sealed graph. Therefore it would be wrong to say that the 30.759-second cold run has become a 0.225-second whole-model run.

The next ambitious step is a **native schema and IR bootstrap**. It should read the reviewed utility specification and skim metadata, allocate the GPU interface, and launch the Phase 29 source compiler without running the dense preprocessor at all. The legacy ActivitySim path would then run only in a separate verification process. The same phase should attack the remaining 57-case frozen boundary map with one shared Sharrow/CUDA arithmetic definition.

86. Phase 30: build the recipe without making the answer sheet

Phase 29 could create every input from real upstream causes, but its live test still let ActivitySim prepare the huge dense table first. That table was not read by the input compiler, yet it was still used to reveal the shape of the GPU interface. It was like writing a recipe independently but still looking at a finished cake to learn how many bowls to put on the counter.

Phase 30 creates the interface without making the finished cake. It starts with four kinds of information:

- a reviewed **intermediate representation**, or IR, containing the 315 equation terms and their 21 travel-mode alternatives;
- a type contract naming the 10 floating-point and 31 integer row sources;
- 48 scalar settings such as constants and coefficients; and
- 209 logical skim sources with six groups of origin, destination, and time coordinates.

Together these form an **application binary interface**, or ABI. ABI sounds complicated, but it simply means a precise agreement about what data a program receives, its type, its order, and its shape. If the recipe asks for an unknown source or a skim cube with the wrong number of dimensions, compilation stops. There is no hidden rule that guesses.

87. What the live native path actually does

For each work, school, or university scheduling program, the live path now:

1. reads and hashes the reviewed utility recipe;
2. takes one raw row per tour plus land-use facts;
3. requests the same controlled random numbers once and preserves their order;
4. uploads immutable raw network skim cubes without creating target rows;
5. compiles named inputs and skim coordinates into the strict CUDA ABI;
6. runs utility, nested logit, cache scatter, scheduling choice, and timetable mutation on the GPU; and
7. publishes only the final schedule labels.

The ordinary ActivitySim workflow still owns model orchestration, raw tables, configuration files, and final publication. Therefore Phase 30 is not a claim that the entire regional model uses no CPU. The precise claim is that the mandatory-tour mode-logsum and scheduling production path no longer executes or reads ActivitySim's dense chooser-alternative preprocessor.

88. How we proved that removing the old path changed nothing

Correct final schedules are necessary, but they are not enough. Two slightly different probability calculations can sometimes choose the same answer by luck. Phase 30 therefore uses several layers of evidence:

- three fresh Python and CUDA processes each run five complete resident replays, making 15 full measurements;
- every replay checks all 1,210,124 logsum values bit for bit;
- all 81,983 final scheduled tour times are compared with ActivitySim;
- every process records stable SHA-256 hashes for the work, school, and university IR and ABI schemas;
- a separate native process deliberately downloads and hashes its six logsum arrays only for qualification;
- a separate legacy process does the same after running the old dense preprocessor; and

- every individual hash and the combined hash match exactly.

The shared combined logsum hash is `41ea4ab90d0b47595a6ad59b1598a050a09a01db5d775dd3d1ad9f5be79e1322`. The deliberate hash downloads are not inside any timed resident run. This separation prevents a correctness test from being quietly counted as GPU performance.

89. The Phase 30 measured result

Result	Process 1	Process 2	Process 3	Middle result
Complete native-input-to-calendar graph	0.226712 s	0.221389 s	0.231066 s	**0.226712 s**
Native ABI bootstrap for six programs	4.540633 s	4.488558 s	4.867005 s	**4.540633 s**
Checkpoint-to-result ActivitySim run	30.222 s	30.779 s	30.739 s	**30.739 s**

The compact persistent facts still occupy 24,849,394 bytes instead of 503,411,584 bytes of repeated row arrays, a **20.259-times reduction**. Plan construction reads zero dense preprocessor rows and zero dense preprocessor values. It avoids creating 1,210,124 dense rows in the production bootstrap.

The complete resident graph is 0.622% slower than Phase 29, while the middle cold run is 0.065% faster. Both differences are small enough to treat as ordinary timing variation. It would be misleading to advertise either as an important performance change.

Why did removing more than a million dense rows not make cold startup much faster? A fresh process spends about 12 seconds loading and arranging a 6.452 GB Sharrow skim dataset before this component can run. It also restores the ActivitySim checkpoint and initializes Python, CUDA, configuration, and model state. Phase 30 removed a dependency, but it did not remove those larger costs.

90. Why the 57-entry arithmetic safeguard remains

Turning utility scores into a choice uses exponential functions, additions, division, and a search through cumulative probabilities. A CPU math library and a CUDA math library can round the final bit differently. Usually that does not matter. It matters when a controlled random draw lands extremely close to a probability boundary.

Phase 30 makes the scheduling dot product, probability sum, and search order explicit. It also compiles and tests two GPU exponential policies:

- CUDA 32-bit exponential detected 58 possibly ambiguous public rows; and
- CUDA 64-bit exponential rounded back to 32 bits detected 57.

Both policies produced zero incorrect choices on the frozen six-batch reference artifact. However, the live newly generated logsum path still shows that one boundary correction can be required. Removing the resident 57-entry map would therefore make the claim broader than the evidence. The runtime keeps the small fail-closed map on the GPU, downloads zero boundary bytes, and states that its exact guarantee applies to this qualified public benchmark.

91. What Phase 30 means and the next ambitious target

Phase 30 is the point where the production GPU program can stand on its own at the mode-logsum bootstrap boundary. Phases 1 through 29 are not made moot. They supply the independent CPU answer keys, shared arithmetic rules, compact data model, raw-source formulas, resident runtime, changed-world tests, and failure gates that make the new independence believable.

The next major opportunity is a persistent native skim store. Instead of loading and rearranging 6.452 GB through Sharrow for every fresh process, a Phase 31 runtime should create a versioned GPU-ready skim artifact once, map or stream only the 149 required physical cubes, verify their content hashes, and reuse them across scenarios and processes. The benchmark must separately time cold file-to-GPU startup, warm scenario reuse, resident computation, and final publication.

In parallel, a shared correctly specified exponential and reduction implementation should run in both the Sharrow reference and CUDA backend. New changed-scenario and boundary-fuzz tests must pass before the 57-entry map can be deleted. After those two frontiers, the same native ABI pattern can expand to non-mandatory tours, joint tours, destinations, trips, and eventually a larger end-to-end device-resident ActivitySim workflow.

92. Phase 31: turn the road atlas into a ready-to-use GPU book

Phase 30 could build the GPU calculation without ActivitySim's giant prepared answer table. But it still asked Sharrow to open and arrange the network measurements every time a new Python process started. Imagine a delivery driver who owns a carefully planned route but must rebuild the whole road atlas from hundreds of compressed pages before every delivery. The route is fast; rebuilding the atlas is not.

The public network file is called an **OMX file**. It contains 826 matrices. A matrix is a rectangular grid of numbers. In this case, rows are origin zones, columns are destination zones, and a number may mean distance, travel time, fare, waiting time, or another travel condition. Many measurements also have five time-of-day versions.

The reviewed mandatory tour equations do not need 826 independent matrices. They make 209 logical requests, but some requests are reverse views or are repeated by work, school, and university programs. After those duplicates are shared, the GPU needs 149 physical cubes. A **cube** here means either one origin-by-destination grid or five such grids stacked by time period.

Phase 31 builds those 149 cubes once and writes them in exactly the order and 32-bit format the GPU runtime expects. This new file is the **native skim store**. “Native” means it is already shaped for this GPU interface. “Skim” is the travel-model word for a zone-to-zone measurement. “Store” simply means a saved collection.

93. What is saved, and what must be trusted

The store has two main pieces:

- `payload.f32` holds 6,198,588,112 bytes of actual 32-bit measurements; and
- `manifest.json` is a detailed contents label.

The label records the 149 names, shapes, dimensions, byte locations, source files, five time periods, 1,454-zone order, and the reviewed 209-request contract. It also records cryptographic fingerprints called **SHA-256 hashes**. A hash is like a very sensitive digital fingerprint: changing even one byte almost certainly changes the fingerprint.

The original compressed OMX file is about 734 million bytes. The ready-to-use store is about 6.20 billion bytes, or 8.446 times larger. This is an honest trade. The project uses more disk space so each model process can avoid decompressing, naming, rearranging, converting, and joining the same data. The store is generated locally rather than checked into Git because a 6.20 GB file would make the source repository impractical.

Before the GPU can trust a store, the loader checks:

1. the file-format version and the fingerprint of the full label;
2. that the reviewed equations still ask for the same skim contract;
3. that the zone identities and their order match;
4. that all cube shapes, byte locations, and types are sensible;
5. that the file fits the declared 8 GiB budget; and
6. the SHA-256 fingerprint over every one of the 6,198,588,112 payload bytes.

If a byte is corrupt, the model recipe changed, the zones moved, or the store is too large, loading stops. Tests deliberately exercise all of those failure paths. This is what **fail closed** means: uncertainty produces an error, not a possibly believable but unproven travel forecast.

94. How the fast loader works

GPU transfers are fastest from a special kind of main memory called **pinned memory**. The operating system agrees not to move those memory pages while the GPU is using them. Phase 31 keeps two reusable pinned buffers:

```
CPU reads and hashes chunk 2  ---------
                                       \
GPU uploads chunk 1          ----------> work overlaps

CPU reads and hashes chunk 3  ---------
                                       \
GPU uploads chunk 2          ----------> work overlaps again
```

This is called **double buffering**. While the GPU uploads one block, the CPU fills and checks the other. The first correct loader did the work mostly one step at a time and hashed the same valid data twice. It took 11.148 seconds to load the store, and the whole cold interval was worse than Phase 30. That failure was measured rather than hidden.

The final loader reads straight from the file into the pinned buffer, makes one ordered fingerprint over every byte, and overlaps the two kinds of work. Its three verified load times are 4.229, 4.043, and 4.320 seconds. The middle result is **4.229 seconds**. A separate 0.074-second number in the report is only upload waiting and bookkeeping left visible after overlap; it is not a claim that 6.20 GB crossed the physical bus in 0.074 seconds.

95. The hidden calls that also had to disappear

Changing one skim function was not enough. Testing revealed three less obvious paths:

- ActivitySim still opened the OMX file to inventory all matrix names;
- its network-data loader still tried to build the Sharrow dataset; and
- 57 choices extremely close to probability boundaries still called Sharrow for a final decision.

Phase 31 bypasses the first two operations only inside this guarded one-zone native runner. The manifest becomes the authoritative inventory. The 57 close choices use the already-qualified answer map stored on the GPU. A conservative setup pass may reserve 58 map positions, but exactly 57 were exercised in each live run. All 57 stayed on the GPU, and zero boundary logsum bytes returned to the CPU. If a new scenario creates an unrecognized close case, the runtime still stops rather than guessing.

The zone check also needed care. The public source calls its zones 1 through 1454, while ActivitySim's saved internal table calls the same ordered rows 0 through 1453. The adapter adds one only when the internal index is exactly that complete consecutive sequence. The reconstructed public IDs must then match the saved zone fingerprint. This is a checked translation, not an assumption that every unfamiliar zone system starts at one.

96. The Phase 31 result and its meaning

The proof uses three fresh Python processes and five full resident replays in each, for 15 complete replays:

Result	Process 1	Process 2	Process 3	Middle result
ActivitySim checkpoint-to-result interval	26.434 s	26.570 s	26.350 s	**26.434 s**
Cold interval including scheduler/map setup	27.085 s	27.167 s	26.933 s	**27.085 s**
Verified native-store load	4.229 s	4.043 s	4.320 s	**4.229 s**
Complete resident raw-source-to-calendar graph	0.217578 s	0.217801 s	0.224729 s	**0.217801 s**

For the fairest conservative comparison, Phase 31 includes scheduler/map setup even though Phase 30's published 30.739-second interval began after its scheduler object was created. Phase 31 still saves **3.654 seconds**, cuts the

measured cold component interval by **11.887%**, and is **1.135 times faster**. Every Phase 31 process beats the Phase 30 middle time.

Correctness did not become approximate:

- every one of 1,210,124 logsum values matches bit for bit in all 15 replays;
- all 81,983 scheduled tour times match;
- every store byte is checked in every process;
- no live Sharrow skim dataset or OMX inventory is built;
- all exercised close-boundary decisions remain on the GPU; and
- Phase 31, Phase 30 native, and Phase 30 legacy calculations share the exact same six-program logsum fingerprint: `41ea4ab90d0b47595a6ad59b1598a050a09a01db5d775dd3d1ad9f5be79e1322`.

What does this mean in the larger project? Phase 31 proves that changing the data-loading architecture can finally make a visible dent in cold component time without giving back the replication guarantees built in earlier phases. Phases 1 through 30 were not wasted: they supplied the answer keys, arithmetic rules, source contracts, device runtime, changed-world tests, and failure checks needed to trust this result.

What does it not mean? It does not make all of ActivitySim GPU-only. It does not say a full regional-model run is 1.135 times faster. It covers the resumed public mandatory-scheduling component on one RTX A4000, one zone system, one network dataset, and one local storage environment. Building or changing the network requires rebuilding and requalifying the store.

The next ambitious job is model-wide reuse. The same compiled-asset registry, native schema, skim store, and device-resident runtime should cover non-mandatory and joint tours, destinations, trip mode choice, and trip scheduling. That phase must publish end-to-end model wall time so component wins are not mistaken for total-model wins. Separately, CPU and GPU still need one shared, precisely specified exponential/reduction implementation plus changed-scenario boundary fuzzing before the small frozen answer map can be removed for every possible scenario.